

Department of Artificial Intelligence and Mathematical Sciences



Lab Manual

Natural Language Processing

Course Code: AI-2201

NAME: Muhammad Yahya

ID: BAI-23F-030

SECTION: 6th(B)

DEPARTMENT: AIMS

Lab Instructor: Muhammad Usman

Sindh Madressatul Islam University, Karachi



CERTIFICATE

This is to certify that **Mr. Muhammad Yahya SO Muhammad Hussain** having Roll No. **BAI-23F-030** has successfully completed laboratory work during Spring Semester 2026.

Teacher Sign:

<u>S. No</u>	<u>LABS</u>	<u>Page No</u>
01	LAB # 01 Introduction to NLP & Text Preprocessing • What is NLP • NLP Pipeline • Tokenization (Word & Sentence) • Removing Punctuation	
02	• Lab # 02: Text Cleaning Techniques • Stop Words Removal • Lowercasing • Removing Special Characters • Text Normalization	
03	Lab # 03: Stemming & Lemmatization • Stemming • Lemmatization • Difference between Stemming and Lemmatization	
04	• Lab # 04: Part of Speech (POS) Tagging • Introduction to POS Tagging • Identifying Nouns, Verbs, Adjectives • Practical Implementation	
05	• Lab # 05: Named Entity Recognition (NER) • Introduction to NER • Extracting Person Names • Extracting Location Names • Extracting Organization Names	
06	Lab # 06: Text Representation Techniques • Bag of Words (BoW) • Count Vectorizer • Feature Extraction	
07	• Lab # 07: TF-IDF Implementation • Term Frequency (TF) • Inverse Document Frequency (IDF) • TF-IDF Vectorization	
08	• Lab # 08: Sentiment Analysis • Introduction to Sentiment Analysis • Rule-Based Sentiment Analysis • Positive and Negative Text Classification	
09	Lab # 09: Text Classification using Machine Learning • Dataset Preparation • Train-Test Split • Naive Bayes Classifier	

10	Lab # 10: NLP Pipeline using Scikit-learn • Creating NLP Pipeline • Vectorizer and Classifier • Model Evaluation (Accuracy)	
-----------	--	--

11	• Lab # 11: Word Embeddings • Introduction to Word Embeddings • Word2Vec Concept • Using Pretrained Embeddings	
12		
	Lab # 12: Mini NLP Project • Text Cleaning • Feature Extraction • Model Training • Model Testing and Result Analysis	

Lab Experiment # 01

Title: Introduction to Natural Language Processing (NLP) and Text Preprocessing by Removing Punctuation

Objective:

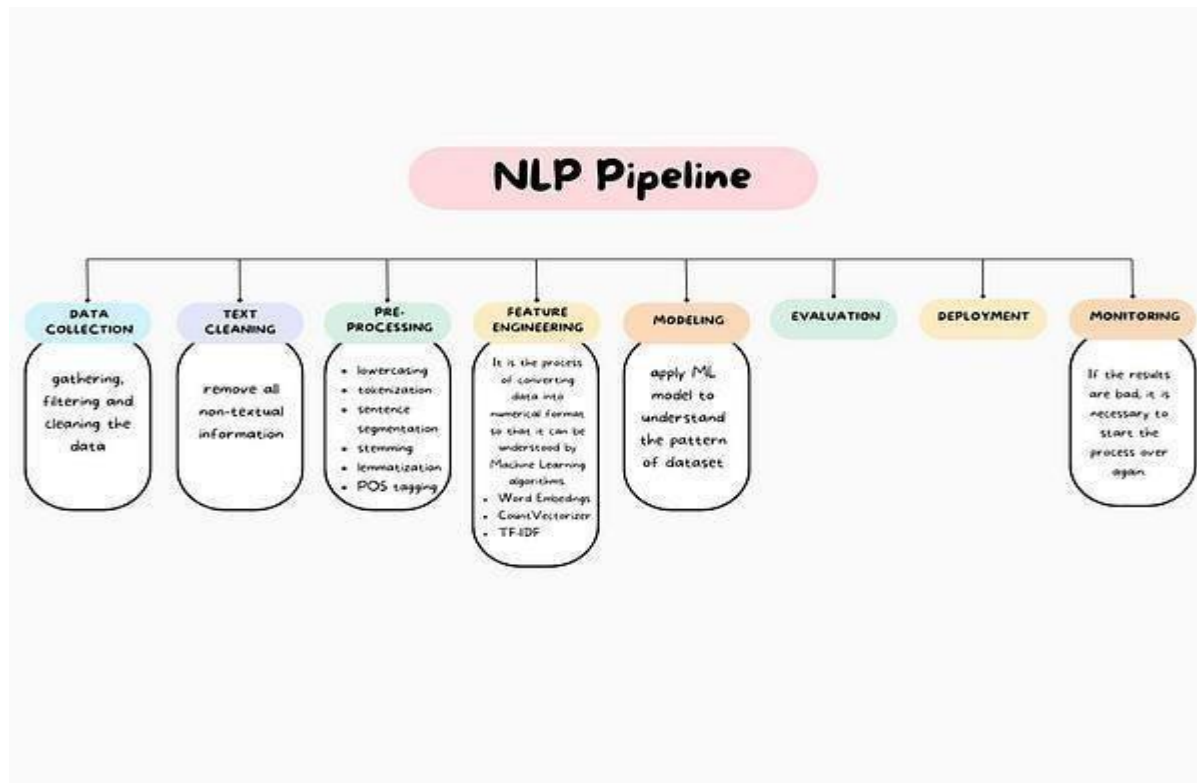
- To understand the concept of **Natural Language Processing (NLP)** and its real-life applications such as voice assistants, chatbots, and text translation.
- To learn the importance of **text preprocessing in NLP**, especially removing punctuation to clean data and improve text analysis accuracy.

What is NLP ?

Natural Language Processing (NLP) is a branch of AI that enables computers to read, understand, and interpret human language.

Real life examples of NLP .

Voice assistants , chatbot , translation of text through software's



Removing Punctuation in NLP

Removing punctuation means deleting symbols like:

., ! ? : ; ' " () from text before

processing it. Example:

Original sentence:

"Hello, how are you?"

After removing punctuation:

Hello how are you

□ **Cleaner Text Data**

Removing punctuation makes the text **clean and easier to process**.

□ **Reduces Noise in Data**

Symbols like ! , . ? do not usually add meaning, so removing them **reduces unnecessary information**.

□ **Better Word Matching**

Words like “hello” and “hello!” become the same after removing punctuation.

□ **Improves Model Accuracy**

The model focuses on **important words instead of symbols**, which can improve results.

□ **Easier Tokenization**

Splitting sentences into words becomes **simpler and more accurate**.

□ **Reduces Vocabulary Size**

Without punctuation, the number of **different tokens (words)** becomes smaller.

Tokenization :

Tokenization is the process of breaking down the text into smaller units called tokens, usually words or phrases.

***Example:** The sentence “I love NLP” would be tokenized into [“I”, “love”, “NLP”]. Each word becomes a separate token.*

Lab Task:**1: Install python jupyter notebook and install the nltk pandas**

```
[1]: !pip install nltk pandas

Requirement already satisfied: nltk in c:\users\kk\anaconda3\lib\site-packages (3.9.2)
Requirement already satisfied: pandas in c:\users\kk\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: click in c:\users\kk\anaconda3\lib\site-packages (from nltk) (8.2.1)
Requirement already satisfied: joblib in c:\users\kk\anaconda3\lib\site-packages (from nltk) (1.5.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\kk\anaconda3\lib\site-packages (from nltk) (2025.9.1)
Requirement already satisfied: tqdm in c:\users\kk\anaconda3\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: numpy>=1.26.0 in c:\users\kk\anaconda3\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\kk\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\kk\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\kk\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\kk\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: colorama in c:\users\kk\anaconda3\lib\site-packages (from click->nltk) (0.4.6)
```

2: take a paragraph of 100 words or 200 words and words then remove the punctuation from the paragraph

```
[2]: import string

paragraph = """Artificial intelligence is transforming the world rapidly. It is used in healthcare, education, and business! Many companies invest heavily in AI hoping to improve efficiency and accuracy. However, challenges remain such as data privacy and ethical concerns. Researchers continue to explore new methods and technologies to make AI more reliable and accessible. In the future, AI will play an even bigger role in our daily lives, helping people solve complex problems and make better decisions."""

# Remove punctuation
clean_text = paragraph.translate(str.maketrans('', '', string.punctuation))

print(clean_text)
```

Artificial intelligence is transforming the world rapidly It is used in healthcare education and business Many companies invest heavily in AI hoping to improve efficiency and accuracy However challenges remain such as data privacy and ethical concerns Researchers continue to explore new methods and technologies to make AI more reliable and accessible In the future AI will play an even bigger role in our daily lives helping people solve complex problems and make better decisions

```
from nltk.tokenize import word_tokenize

text = "Artificial intelligence is transforming the world rapidly."
tokens = word_tokenize(text)

print(tokens)
```

```
... ['Artificial', 'intelligence', 'is', 'transforming', 'the', 'world', 'rapidly', '.']
```

3: Perform tokenization of the paragraph taken

Lab Experiment # 02

Title: Introduction to Natural Language Processing (NLP) and Text Preprocessing Techniques

Objective:

- To understand the basic concept of Natural Language Processing (NLP) and its real-life applications such as voice assistants, chatbots, and translation systems.
- To learn different text preprocessing techniques like lowercasing, removing punctuation, special characters, and stop words to clean and prepare text data for analysis.

Stop Words Removal

Stop words are very common words that usually do not add important meaning to a sentence. Examples include: **is, the, a, an, in, on, and, of**

These words are removed to make the text smaller and easier to analyze.

Example

Original sentence:

This is a book on Natural Language Processing

After removing stop words:

book Natural Language Processing

Lowercasing

Lowercasing means converting all letters in the text to **small letters**.

This helps the computer treat words like **Hello, HELLO, hello** as the same word.

Example

Original text:

Hello World

After lowercasing:

hello world

Removing Special Characters

Special characters are symbols that usually do not add meaning in text analysis.

Examples include:

@ # \$ % ^ & * () _ +

Removing them makes the text cleaner.

Example

Original sentence:

Welcome!!! to NLP @ 2025

After removing special characters:

Welcome to NLP 2025

Text Normalization

Text normalization means converting text into a **standard and consistent form** so that it becomes easier for the computer to understand.

It may include:

- Lowercasing
- Removing punctuation
- Removing special characters
- Expanding short forms

Example

Original text:

I'm learning NLP!!!

Normalized text:

i am learning nlp

Lab Task

Write a Python program to perform the following **text preprocessing steps**:

1. Take a sentence as input.
2. Convert the text into **lowercase**.
3. Remove **punctuation and special characters**.
4. Remove **stop words** from the sentence.
5. Display the **cleaned text** after preprocessing.

Example Input:

Hello! I am learning Natural Language Processing.

Expected Output:

learning natural language processing

```

import string
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

text = "Hello! I am learning Natural Language Processing."

# Lowercase
text = text.lower()

# Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))

# Tokenize
words = word_tokenize(text)

# Stopwords
stop_words = set(stopwords.words('english'))

# Add custom word to remove
stop_words.add('hello')

filtered_words = [word for word in words if word not in stop_words]

clean_text = " ".join(filtered_words)

print(clean_text)

```

```

... learning natural language processing

```

Lab Experiment # 03

Title: Introduction to Natural Language Processing (NLP) and Text Preprocessing Techniques

Objective:

- To understand the basic concept of Natural Language Processing (NLP) and its real-life applications

Stemming

Stemming is a text preprocessing technique that **reduces words to their root or base form** by chopping off suffixes.

It may not produce a real word but helps in grouping similar words together.

Example:

- running → run
- studies → studi
- happily → happi

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
words = ["running", "studying", "smiling",
"communication"] stemmed_words =
[stemmer.stem(word) for word in words]
print(stemmed_words)
outputs : ['run', 'studi', 'smile', 'commun']
```

Lemmatization(grammar)

Lemmatization also reduces words to their root form, but it **produces a valid word (lemma)** using dictionary knowledge and context.

Example:

- running → run
- studies → study
- better → good

```
from nltk.stem import
```

```
WordNetLemmatizer from nltk.corpus  
import wordnet
```

```
lemmatizer = WordNetLemmatizer()

words = ["running", "better", "studies",
"was"] lemmas = [
    lemmatizer.lemmatize("running",
    pos=wordnet.VERB),
    lemmatizer.lemmatize("better",
    pos=wordnet.ADJ),
    lemmatizer.lemmatize("studies",
    pos=wordnet.NOUN),
    lemmatizer.lemmatize("was",
    pos=wordnet.VERB)
]

print(lemmas)
```

Lab Task:

- Remove punctuation from the paragraph (200 words)
- Covert into lower
- Turn the paragraph into tokens
- Perform stemming and lemmatization on a paragraph or 200 words

```
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Downloads (run once)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Input paragraph
text = """Artificial intelligence is rapidly transforming the world. It is being used in healthcare, education, transportation, and business industries. Many companies are investing heavily in AI research and development. The growth of artificial intelligence is expected to continue at a rapid pace over the next several years.""""

# Step 1: Lowercase
text = text.lower()

# Step 2: Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))

# Step 3: Tokenization
tokens = word_tokenize(text)

# Step 4: Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word not in stop_words]
print("Tokens after preprocessing:\n", filtered_tokens)

# Step 5: Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_tokens]
print("\nStemmed Words:\n", stemmed_words)

# Step 6: Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]
print("\nLemmatized Words:\n", lemmatized_words)
```

```
*** [nlk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
Tokens after preprocessing:
['artificial', 'intelligence', 'rapidly', 'transforming', 'world', 'used', 'healthcare', 'education', 'transportation', 'business', 'industries', 'many', 'companies', 'invest', 'heavily', 'ai', 'research', 'development', 'growth', 'continue', 'pace', 'years']

Stemmed Words:
['articifi', 'intellig', 'rapidli', 'transform', 'world', 'use', 'healthcar', 'educ', 'transport', 'busi', 'industri', 'mani', 'compani', 'invest', 'heavili', 'ai', 'reserch', 'developm', 'growt', 'contin', 'pac', 'year']

Lemmatized Words:
['artificial', 'intelligence', 'rapidly', 'transforming', 'world', 'used', 'healthcare', 'education', 'transportation', 'business', 'industry', 'many', 'company', 'investment', 'heavily', 'ai', 'research', 'development', 'growth', 'continue', 'pace', 'years']
```


Lab Experiment # 04

Title: Introduction to Natural Language Processing (NLP) and Text Preprocessing Techniques

Objective:

To understand and implement Part-of-Speech (POS) Tagging in Python to identify grammatical categories of words in a sentence.

Post tagging:

POS Tagging (Part-of-Speech Tagging) is a process in Natural Language Processing (NLP) where each word in a sentence is assigned a grammatical category.

These categories help the computer understand the structure and meaning of a sentence.

Tag	Meaning	Example
NN	Noun	Ali, Car
VB	Verb	Run, Eat
JJ	Adjective	Beautiful
RB	Adverb	Quickly
PRP	Pronoun	He, She
DT	Determiner	The, A

Tag	Meaning	Example
NN	Noun	Ali, Car
VB	Verb	Run, Eat
JJ	Adjective	Beautiful
RB	Adverb	Quickly
PRP	Pronoun	He, She
DT	Determiner	The, A

Code:

Required Libraries

```
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
```

```
import nltk
from nltk.tokenize import
word_tokenize from nltk import
pos_tag
```

```
sentence = "Ali is playing football in the park"
```

Tokenization

```
words = word_tokenize(sentence)
```

POS Tagging

```
pos_tags = pos_tag(words)
```

```
print(pos_tags)
```

CODE FOR POS Tagging

```
import nltk
```

```
from nltk.tokenize import
```

```
word_tokenize from nltk import
```

```
pos_tag
```

```
sentence = "Ali is playing football in the  
park" # Tokenization
```

```
words = word_tokenize(sentence)
```

```
# POS Tagging
```

```
pos_tags = pos_tag(words)
```

```
print(pos_tags)
```

OUTPUT

```
[('Ali', 'NNP'),  
( 'is', 'VBZ'),  
( 'playing', 'VBG'),  
( 'football', 'NN'),  
( 'in', 'IN'),  
( 'the', 'DT'),  
( 'park', 'NN')]
```

Lab 4 task

1. Write a Python program to perform POS tagging .
- "Sara is reading a book." Write your own sentences

```
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng') # NEW FIX for Colab

from nltk.tokenize import word_tokenize
from nltk import pos_tag

sentence = "The students are learning natural language processing."

words = word_tokenize(sentence)
tags = pos_tag(words)

print(tags)
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[('The', 'DT'), ('students', 'NNS'), ('are', 'VBP'), ('learning', 'VBG'), ('natural', 'JJ'), ('language', 'NN'), ('processing', 'NN'), ('.', '.')]

```

2. Take a sentence input from the user and display POS tags for each word.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Downloads (run once in Colab)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')

# Take input from user
sentence = input("Enter a sentence: ")

# Tokenize
words = word_tokenize(sentence)

# POS tagging
tags = pos_tag(words)

# Output
print("POS Tags:\n", tags)
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
Enter a sentence: I am learning NLP in university.
POS Tags:
[('I', 'PRP'), ('am', 'VBP'), ('learning', 'VBG'), ('NLP', 'NNP'), ('in', 'IN'), ('university', 'NN'), ('.', '.')]

```

3. Write a program that counts the number of nouns and verbs in a sentence. (counter function will be used) from collections import Counter

```

import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from collections import Counter
# Downloads (run once in Colab)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
# Input sentence
sentence = input("Enter a sentence: ")
# Tokenization
words = word_tokenize(sentence)
# POS tagging
tags = pos_tag(words)
# Count all POS tags
tag_counts = Counter(tag for word, tag in tags)
# Count nouns
nouns = (
    tag_counts['NN'] + tag_counts['NNS'] + tag_counts['NNP'] + tag_counts['NNPS']
)
# Count verbs
verbs = (
    tag_counts['VB'] + tag_counts['VBD'] + tag_counts['VBG'] + tag_counts['VBN'] + tag_counts['VBP'] + tag_counts['VBZ']
)
print("POS Tags:", tags)
print("Number of Nouns:", nouns)
print("Number of Verbs:", verbs)

```

```

... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
Enter a sentence: Students are learning Python programming in the classroom.
POS Tags: [('Students', 'NNS'), ('are', 'VBP'), ('learning', 'VBG'), ('Python', 'NNP'), ('programming', 'NN'), ('in', 'IN'), ('the', 'DT'), ('classroom', 'NN'), ('.', '.')]
Number of Nouns: 4
Number of Verbs: 2

```

4. Perform POS tagging on the sentence:

- "The quick brown fox jumps over the lazy dog."
- Identify:
- Nouns , Verbs , Adjectives

```

import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
# Downloads (run once in Colab)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
# Sentence
sentence = "The quick brown fox jumps over the lazy dog."
# Tokenization
words = word_tokenize(sentence)
# POS tagging
tags = pos_tag(words)
# Lists
nouns = []
verbs = []
adjectives = []
# Classification
for word, tag in tags:
    if tag.startswith("NN"):
        nouns.append(word)
    elif tag.startswith("VB"):
        verbs.append(word)
    elif tag.startswith("JJ"):
        adjectives.append(word)
# Output
print("POS Tags:", tags)
print("\nNouns:", nouns)
print("Verbs:", verbs)
print("Adjectives:", adjectives)

```

```

... POS Tags: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('.', '.')]

Nouns: ['brown', 'fox', 'dog']
Verbs: ['jumps']
Adjectives: ['quick', 'lazy']

```

Lab Experiment # 05

Title: Introduction to Natural Language Processing (NLP) and Text Preprocessing Techniques

Objective:

To identify and classify named entities such as **Person, Location, and Organization** from text using Python.

Introduction

Named Entity Recognition (NER) is a task in **Natural Language Processing (NLP)** that identifies important entities in a sentence and classifies them into categories such as:

- **Person**
- **Location**
- **Organization**
- **Date**
- **Time**

NER helps computers understand **who, where, and what** is mentioned in the text. NER is used in many applications such as:

- Search engines
- Chatbots
- News analysis
- Information extraction

"Elon Musk is the CEO of Tesla and he lives in the United States." NER Output:

Word	Entity Type
Elon Musk	Person
Tesla	Organization
United States	Location

Installation libraries

- ❑ `pip install spacy` → installs the spaCy library used for Named Entity Recognition.
- ❑ `pip install nltk` → installs the NLTK library for other NLP tasks like tokenization and POS tagging.
- ❑ `pip install notebook` → installs Jupyter Notebook to run Python code interactively.

- `python -m spacy download en_core_web_sm` → downloads the English language model required for NER.

Required Libraries (Installation Steps)

```
import spacy          # Import the spaCy library used for
NLP tasks # Load the English language model
nlp = spacy.load("en_core_web_sm")
```

```
# Define a sentence that we want to analyze
text = "Elon Musk founded Tesla in the United States."
```

```
# Process the text using the spaCy NLP
pipeline doc = nlp(text)
```

```
# Loop through all detected named entities in the
sentence for entity in doc.ents:
```

```
    # Print the entity text and its entity
    label print(entity.text, entity.label_)
```

OUTPUT

Elon Musk PERSON

Tesla ORG

United States GPE

Function / Method	Description	Example Use
<code>import spacy</code>	Imports the spaCy library for NLP tasks	<code>import spacy</code>
<code>spacy.load()</code>	Loads a trained language model	<code>nlp = spacy.load("en_core_web_sm")</code>
<code>nlp()</code>	Processes the text using the NLP pipeline	<code>doc = nlp(text)</code>
<code>doc.ents</code>	Returns all named entities detected in the text	<code>for ent in doc.ents:</code>
<code>ent.text</code>	Displays the actual entity word or phrase	<code>print(ent.text)</code>
<code>ent.label_</code>	Displays the category of the entity (PERSON, ORG, etc.)	<code>print(ent.label_)</code>
<code>spacy.displacy.render()</code>	Visualizes named entities in the text	<code>displacy.render(doc, style="ent")</code>

Task 1: Perform NER on a Simple Sentence

Sentence: "Bill Gates founded Microsoft in the United States." **Steps:**

1. Import the **spaCy** library.
2. Load the **English language model** (`en_core_web_sm`).
3. Store the sentence in a variable.
4. Process the sentence using the **nlp()** function.
5. Print all detected **named entities and their labels**.

```

import spacy
# Load English model
nlp = spacy.load("en_core_web_sm")
# Sentence
sentence = "Bill Gates founded Microsoft in the United States."
# Process text
doc = nlp(sentence)
# Print entities
for ent in doc.ents:
    print(ent.text, "-", ent.label_)

```

```

*** Bill Gates - PERSON
    Microsoft - ORG
    the United States - GPE

```

Task 2: NER on User Input

Steps:

1. Import **spaCy**.
2. Load the English model.
3. Ask the user to **enter a sentence** using `input()`.
4. Process the text using **nlp()**.
5. Print each **entity and its**

type. Example:

Enter a sentence: Elon Musk founded SpaceX in California.

```

import spacy

nlp = spacy.load("en_core_web_md")

sentence = input("Enter a sentence: ")

doc = nlp(sentence)

print("\nNamed Entities:\n")
for ent in doc.ents:
    print(ent.text, "-", ent.label_)

```

```

*** Enter a sentence: Elon Musk founded SpaceX in California.

Named Entities:

Elon Musk - PERSON
SpaceX - ORG
California - GPE

```

Task 3: Identify Specific Entity Types

Sentence: "Imran Khan was born in Lahore and served as Prime Minister of Pakistan." **Steps:**

1. Import spaCy.
2. Load the English model.
3. Store the sentence in a variable.
4. Extract entities using **doc.ents**.
5. Print entities that belong to:
 - **PERSON**
 - **LOCATION / GPE**
 - **ORGANIZATION / ORG**

```

import spacy

nlp = spacy.load("en_core_web_md")

sentence = "Imran Khan was born in Lahore and served as Prime Minister of Pakistan."

doc = nlp(sentence)

for ent in doc.ents:
    if ent.label_ in ["PERSON", "GPE", "ORG"]:
        print(ent.text, "-", ent.label_)

```

```

*** Imran Khan - PERSON
    Lahore - GPE
    Pakistan - GPE

```

Task 4: Count Total Named Entities

Sentence: "Apple released the iPhone in California." **Steps:**

1. Import spaCy.
2. Load the English model.
3. Store the sentence in a variable.
4. Process the text using **nlp()**.
5. Use **len(doc.ents)** to count total entities.
6. Print the total number of

entities. Example Output:

Total Entities Found: 2

```

import spacy
# Load English model
nlp = spacy.load("en_core_web_sm")
# Sentence
sentence = "Apple released the iPhone in California."
# Process text
doc = nlp(sentence)
# Count entities
total_entities = len(doc.ents)
# Output
print("Named Entities:", [(ent.text, ent.label_) for ent in doc.ents])
print("Total Named Entities:", total_entities)

```

```

*** Named Entities: [('Apple', 'ORG'), ('iPhone', 'ORG'), ('California', 'GPE')]
    Total Named Entities: 3

```

Task 5: NER on a Paragraph

Paragraph: "Elon Musk is the CEO of Tesla. He lives in the United States and often visits California." **Steps:**

1. Import the spaCy library.
2. Load the English language model.
3. Store the paragraph in a variable.
4. Process the paragraph using `nlp()`.
5. Loop through `doc.ents`.
6. Print all detected entities with their labels.

```
import spacy
# Load English model
nlp = spacy.load("en_core_web_sm")
# Paragraph
paragraph = "Elon Musk is the CEO of Tesla. He lives in the United States and often visits California."
# Process text
doc = nlp(paragraph)
# Print all entities
print("Named Entities:\n")
for ent in doc.ents:
    print(ent.text, "-", ent.label_)
```

```
... Named Entities:
Elon Musk - PERSON
Tesla - ORG
the United States - GPE
California - GPE
```

Lab Experiment # 06

Title: Introduction to Bag of Words (BoW) and N-Grams in Natural Language Processing

Objective:

To understand and implement two fundamental text representation techniques in Natural Language Processing (NLP) — Bag of Words (BoW) and N-Grams — using Python's NLTK library. Students will learn how raw text is converted into numerical features and sequences that computers can process

Introduction to Bag of Words

Bag of Words (BoW) is one of the simplest and most widely used techniques for converting text into a numerical form that machine learning models can understand. The idea is straightforward: represent a document as a collection (or 'bag') of the words it contains, ignoring grammar and word order, but keeping track of how many times each word appears.

Imagine tipping all the words of a sentence into a bag and shaking them up. The bag no longer remembers the original order, but it still tells you exactly which words were used and how many times — that is the Bag of Words model.

Why is Bag of Words Necessary?

Computers cannot work directly with raw text. They need numbers. BoW solves this by converting any piece of text into a fixed-length vector of word counts. This makes it possible to:

Compare documents mathematically

Train machine learning classifiers (e.g., spam detection, sentiment analysis) Build search engines and document ranking systems

Extract topics from large text collections

How Bag of Words Works — Step by Step

Consider two sentences:

```
Sentence 1: "I love NLP"  
Sentence 2: "I love Python and NLP"
```

Step 1 — Build a Vocabulary (unique words across all sentences):

Vocabulary: [I, love, NLP, Python, and]

Step 2 — Create a Word-Count Vector for each sentence:

Sentence	I	love	NLP	Python	and
Sentence 1	1	1	1	0	0
Sentence 2	1	1	1	1	1

Each sentence is now a vector of numbers. Machine learning models can work directly with these vectors.

Applications of Bag of Words

Email Spam Detection

Sentiment Analysis (positive / negative reviews) Document Classification

Information Retrieval / Search

Engines Plagiarism Detection

Limitations of Bag of Words

Ignores word order — "dog bites man" and "man bites dog" give the same

BoW Ignores meaning — synonyms are treated as different words

Creates very large, sparse vectors for big vocabularies

Installation & Required Libraries

```
pip install nltk          # installs NLTK for NLP tasks
pip install scikit-learn  # installs scikit-learn (CountVectorizer
for BoW)
pip install notebook      # installs Jupyter Notebook
```

Also run the following inside Python to download NLTK data:

```
import nltk
nltk.download('punkt')      # tokenizer data
nltk.download('stopwords')  # common stopwords
nltk.download('punkt_tab')  # required for newer NLTK versions
```

Sample Code — Bag of Words with NLTK

The following example builds a Bag of Words representation from scratch using NLTK:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from collections import Counter
import string

# Download required NLTK data
nltk.download('punkt')
nltk.download('stopwords')

# Define a sample sentence
text = "Natural Language Processing is a field of Artificial
Intelligence."
```

```
# Step 1: Tokenize the text into individual words
tokens = word_tokenize(text.lower())    # convert to lowercase first
print('Tokens:', tokens)

# Step 2: Remove punctuation
tokens = [t for t in tokens if t not in string.punctuation]

# Step 3: Remove stopwords (common words like 'is', 'a', 'the')
stop_words = set(stopwords.words('english'))
filtered = [t for t in tokens if t not in stop_words]
print('Filtered tokens:', filtered)

# Step 4: Count word frequencies (Bag of Words)
bow = Counter(filtered)
print('Bag of Words:', dict(bow))
```

Expected Output:

```
Tokens: ['natural', 'language', 'processing', 'is', 'a', 'field',
'of', 'artificial', 'intelligence', '.']
Filtered tokens: ['natural', 'language', 'processing', 'field',
'artificial', 'intelligence']
Bag of Words: {'natural': 1, 'language': 1, 'processing': 1,
'field': 1, 'artificial': 1, 'intelligence': 1}
```

NLTK Function / Method Reference — Bag of Words

Function / Method	Description	Example
<code>word_tokenize(text)</code>	Splits text into a list of individual word tokens	<code>word_tokenize('I love NLP')</code>
<code>text.lower()</code>	Converts all characters to lowercase for consistency	<code>'NLP'.lower() → 'nlp'</code>
<code>stopwords.words('english')</code>	Returns a list of common English stopwords	<code>set(stopwords.words('english'))</code>

	to remove	
string.punctuation	A string of punctuation characters	if t not in string.punctuation

Function / Method	Description	Example
	used for filtering	
<code>Counter(list)</code>	Counts the frequency of each element in a list	<code>Counter(['a','b','a']) → {'a':2,'b':1}</code>
<code>dict(counter)</code>	Converts Counter object to a plain Python dictionary	<code>dict(bow)</code>
<code>CountVectorizer()</code>	Builds a full BoW matrix from a list of documents (scikit-learn)	<code>vec.fit_transform(docs)</code>
<code>fit_transform(docs)</code>	Learns vocabulary and converts documents to a BoW matrix	<code>X = vec.fit_transform(corpus)</code>
<code>get_feature_names_out()</code>	Returns the vocabulary words learned by CountVectorizer	<code>vec.get_feature_names_out()</code>

Part B: N-Gram

Introduction to N-Grams

An N-Gram is a contiguous sequence of N items (words, characters, or tokens) extracted from a piece of text. Unlike Bag of Words which treats each word independently, N-Grams capture the order and context of words, making them more powerful for many NLP tasks.

The letter N simply refers to the number of items in each sequence:

N-Gram Type	Value of N	Example from "I love NLP"
Unigram	1	['I'], ['love'], ['NLP']
Bigram	2	['I love'], ['love NLP']
Trigram	3	['I love NLP']
4-gram	4	(needs at least 4 words)

Why are N-Grams Necessary?

Bag of Words completely ignores word order. This causes problems in many real-world scenarios. N-Grams solve this by capturing sequences of consecutive words, preserving some local context. For example:

"not good" is very different from "good" — a unigram model misses this "New York" should be treated as one entity, not two separate words

"machine learning" as a bigram carries a different meaning than 'machine' + 'learning' separately N-Grams are used in applications such as:

Language Models — predicting the next word in a sentence (e.g., autocomplete) Machine Translation — translating phrases rather than single words

Speech Recognition — matching spoken word sequences Text Generation — generating coherent sentences

Spelling Correction — finding likely next words based on context

Types of N-Grams in Detail

1. Unigrams ($N = 1$)

Each word is treated as a single unit. This is essentially Bag of Words.

```
Sentence: "The cat sat on the mat"  
Unigrams: ['The', 'cat', 'sat', 'on', 'the', 'mat']
```

2. Bigrams (N = 2)

Pairs of consecutive words. Captures simple local context.

```
Sentence: "The cat sat on the mat"
Bigrams: [('The', 'cat'), ('cat', 'sat'), ('sat', 'on'), ('on', 'the'),
          ('the', 'mat')]
```

3. Trigrams (N = 3)

Groups of three consecutive words. Captures richer context than bigrams.

```
Sentence: "The cat sat on the mat"
Trigrams: [('The', 'cat', 'sat'), ('cat', 'sat', 'on'),
          ('sat', 'on', 'the'), ('on', 'the', 'mat')]
```

Character N-Grams

Instead of words, character N-Grams operate on individual characters. They are useful for handling misspellings and out-of-vocabulary words.

```
Word: "hello"
Character bigrams: ['he', 'el', 'll', 'lo']
```

Sample Code — N-Grams with NLTK

The following example shows how to generate unigrams, bigrams, and trigrams using NLTK's built-in `ngrams()` function:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter

nltk.download('punkt')

# Define a sample sentence
text = "The cat sat on the mat near the wall"

# Step 1: Tokenize the sentence
tokens = word_tokenize(text.lower())
print('Tokens:', tokens)

# Step 2: Generate Unigrams (single words)
unigrams = list(ngrams(tokens, 1))
print('\nUnigrams:', unigrams)

# Step 3: Generate Bigrams (pairs of words)
bigrams = list(ngrams(tokens, 2))
print('\nBigrams:', bigrams)
```

```
# Step 4: Generate Trigrams (groups of 3 words)
trigrams = list(ngrams(tokens, 3))
print('\nTrigrams:', trigrams)

# Step 5: Count bigram frequencies
bigram_freq = Counter(bigrams)
print('\nBigram Frequencies:', dict(bigram_freq))
```

Expected Output:

```
Tokens: ['the', 'cat', 'sat', 'on', 'the', 'mat', 'near', 'the', 'wall']

Unigrams: [('the',), ('cat',), ('sat',), ('on',), ('the',), ('mat',), ('near',), ('the',), ('wall',)]

Bigrams: [('the', 'cat'), ('cat', 'sat'), ('sat', 'on'), ('on', 'the'), ('the', 'mat'), ('mat', 'near'), ('near', 'the'), ('the', 'wall')]

Trigrams: [('the', 'cat', 'sat'), ('cat', 'sat', 'on'), ('sat', 'on', 'the'), ('on', 'the', 'mat'), ('the', 'mat', 'near'), ('mat', 'near', 'the'), ('near', 'the', 'wall')]

Bigram Frequencies: {'the', 'cat':1, ('cat', 'sat'):1, ('sat', 'on'):1, ('on', 'the'):1, ('the', 'mat'):1, ('mat', 'near'):1, ('near', 'the'):1, ('the', 'wall'):1}
```

NLTK Function / Method Reference — N-Grams

Function / Method	Description	Example Use
<code>ngrams(tokens, n)</code>	Generates all N-grams of size n from a token list	<code>ngrams(tokens, 2) → bigrams</code>
<code>bigrams(tokens)</code>	Shortcut to generate all bigrams	<code>list(bigrams(tokens))</code>

	(N=2) from tokens	
<code>trigrams(tokens)</code>	Shortcut to generate all trigrams	<code>list(trigrams(tokens))</code>

Function / Method	Description	Example Use
	(N=3) from tokens	
<code>everygrams(tokens, max_len)</code>	Generates all n-grams from unigrams up to max_len	<code>list(everygrams(tokens, 3))</code>
<code>word_tokenize(text)</code>	Tokenizes text into words (required before n-gramming)	<code>word_tokenize(text.lower())</code>
<code>Counter(ngram_list)</code>	Counts frequency of each n-gram tuple	<code>Counter(bigrams)</code>
<code>list(generator)</code>	Converts the ngrams generator object into a list	<code>list(ngrams(tokens, 2))</code>
<code>ConditionalFreqDist()</code>	Computes conditional frequency of word given previous	Used in language modeling

	word	
--	------	--

Lab Tasks

Sentence: "Artificial Intelligence is transforming the world of technology."

Steps:

Import the nltk library and the word_tokenize function. Download the required NLTK data: punkt and stopwords. Convert the sentence to lowercase and tokenize it.

Remove all punctuation from the token list.

Remove stopwords using
nltk.corpus.stopwords.

Count the frequency of each remaining word using Counter. Print the final Bag of Words dictionary.

Expected Output:

```
Tokens: ['artificial', 'intelligence', 'is', 'transforming', 'the',  
'world', 'of', 'technology', '.']  
After stopword removal: ['artificial', 'intelligence',  
'transforming', 'world', 'technology']  
Bag of Words: {'artificial': 1, 'intelligence': 1, 'transforming':  
1, 'world': 1, 'technology': 1}
```

```

▶ # Step 1: Import libraries
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string
from collections import Counter

# Step 2: Download required data
nltk.download('punkt')
nltk.download('stopwords')

# Step 3: Input sentence
sentence = "Artificial Intelligence is transforming the world of technology."

# Step 4: Convert to lowercase and tokenize
tokens = word_tokenize(sentence.lower())

# Step 5: Remove punctuation
tokens = [word for word in tokens if word not in string.punctuation]

# Step 6: Remove stopwords
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word not in stop_words]

# Step 7: Count word frequency
bow = Counter(filtered_tokens)

# Step 8: Print Bag of Words
print("Bag of Words:", dict(bow))

... Bag of Words: {'artificial': 1, 'intelligence': 1, 'transforming': 1, 'world': 1, 'technology': 1}

```

Task 2: BoW on User Input

Write a program that accepts a sentence from the user, then displays its Bag of Words.

Steps:

1. Import nltk, word_tokenize, stopwords, and Counter.
2. Ask the user to enter any sentence using input().
3. Tokenize and convert to lowercase.
4. Remove punctuation and stopwords.
5. Count frequencies with Counter and display the BoW.

Example Interaction:

Enter a sentence: Pakistan is a beautiful country with rich culture.

Bag of Words: {'pakistan': 1, 'beautiful': 1, 'country': 1, 'rich': 1, 'culture': 1}

```
# Step 1: Import libraries
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from collections import Counter
import string

# Ensure required NLTK data is available
for resource in ['punkt', 'punkt_tab', 'stopwords']:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource)

# Step 2: Take input from user
sentence = input("Enter a sentence: ")

# Step 3: Tokenize and convert to lowercase
tokens = word_tokenize(sentence.lower())

# Step 4: Remove punctuation and stopwords
tokens = [word for word in tokens if word not in string.punctuation]

stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word not in stop_words]

# Step 5: Count frequency and display Bag of Words
bow = Counter(filtered_tokens)

print("Bag of Words:", dict(bow))
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Enter a sentence: AI is changing the world of technology!
Bag of Words: {'ai': 1, 'changing': 1, 'world': 1, 'technology': 1}
```

Task 3: Compare BoW of Two Sentences

Sentence A: "Python is great for data science."

Sentence B: "Data science uses Python and statistics." **Steps:**

6. Tokenize both sentences and remove stopwords.
7. Build a combined vocabulary (unique words from both sentences).
8. Create a word-count vector for each sentence based on the vocabulary.
9. Print both vectors side by side.

Expected Output:

Vocabulary: ['data', 'great', 'python', 'science', 'statistics', 'uses']

Sentence A vector: [1, 1, 1, 1, 0, 0]

Sentence B vector: [1, 0, 1, 1, 1, 1]

```
# Step 1: Imports
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import string
# Ensure required data is available
for resource in ['punkt', 'punkt_tab', 'stopwords']:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource)

# Sentences
sentence_a = "Python is great for data science."
sentence_b = "Data science uses Python and statistics."

# Step 6: Tokenize + lowercase + remove punctuation + stopwords
stop_words = set(stopwords.words('english'))
def preprocess(sentence):
    tokens = word_tokenize(sentence.lower())
    tokens = [w for w in tokens if w not in string.punctuation]
    tokens = [w for w in tokens if w not in stop_words]
    return tokens
tokens_a = preprocess(sentence_a)
tokens_b = preprocess(sentence_b)

# Step 7: Build combined vocabulary
vocab = sorted(set(tokens_a + tokens_b))
# Step 8: Create word-count vectors
def vectorize(tokens, vocab):
    return [tokens.count(word) for word in vocab]
vector_a = vectorize(tokens_a, vocab)
vector_b = vectorize(tokens_b, vocab)

# Step 9: Print results
print("Vocabulary:", vocab)
print("Sentence A Vector:", vector_a)
print("Sentence B Vector:", vector_b)
# Side-by-side display
print("\nWord\t\tA\t\tB")
for i, word in enumerate(vocab):
    print(f"{word:12}{vector_a[i]}\t{vector_b[i]}")
```

```
*** Vocabulary: ['data', 'great', 'python', 'science', 'statistics', 'uses']
Sentence A Vector: [1, 1, 1, 1, 0, 0]
Sentence B Vector: [1, 0, 1, 1, 1, 1]
```

Word	A	B
data	1	1
great	1	0
python	1	1
science	1	1
statistics	0	1
uses	0	1

Task 4: Generate Unigrams, Bigrams, and Trigrams

Sentence: "Machine learning helps computers learn from data."

Steps:

10. Import nltk, word_tokenize, and ngrams from nltk.util.
11. Tokenize the sentence.
12. Generate and print unigrams (N=1).
13. Generate and print bigrams (N=2).
14. Generate and print trigrams (N=3).

Expected Output:

```
Unigrams: [('machine',), ('learning',), ('helps',), ('computers',),
('learn',), ('from',), ('data',)]

Bigrams: [('machine', 'learning'), ('learning', 'helps'),
('helps', 'computers'),
('computers', 'learn'), ('learn', 'from'), ('from', 'data')]

Trigrams: [('machine', 'learning', 'helps'),
('learning', 'helps', 'computers'),
('helps', 'computers', 'learn'), ('computers', 'learn', 'from'),
('learn', 'from', 'data')]
```

```
# Step 10: Import libraries
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import ngrams

# Ensure tokenizer is available
for resource in ['punkt', 'punkt_tab']:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource)

# Sentence
sentence = "Machine learning helps computers learn from data."

# Step 11: Tokenize (lowercase for consistency)
tokens = word_tokenize(sentence.lower())

# Step 12: Unigrams (N=1)
unigrams = list(ngrams(tokens, 1))

# Step 13: Bigrams (N=2)
bigrams = list(ngrams(tokens, 2))

# Step 14: Trigrams (N=3)
trigrams = list(ngrams(tokens, 3))

# Print results
print("Tokens:", tokens)
print("\nUnigrams:", unigrams)
print("\nBigrams:", bigrams)
print("\nTrigrams:", trigrams)

... Tokens: ['machine', 'learning', 'helps', 'computers', 'learn', 'from', 'data', '.']

Unigrams: [('machine',), ('learning',), ('helps',), ('computers',), ('learn',), ('from',), ('data',), ('.',)]]

Bigrams: [('machine', 'learning'), ('learning', 'helps'), ('helps', 'computers'), ('computers', 'learn'), ('learn', 'from'), ('from', 'data'), ('data', '.')]

Trigrams: [('machine', 'learning', 'helps'), ('learning', 'helps', 'computers'), ('helps', 'computers', 'learn'), ('computers', 'learn', 'from'), ('learn', 'from', 'data'), ('from', 'data', '.')]

```

Task 5: Find Most Common Bigrams in a Paragraph

Paragraph: "Deep learning is a subset of machine learning. Machine learning is a subset of artificial intelligence. Deep learning uses neural networks."

Steps:

15. Tokenize the paragraph.
16. Generate all bigrams using `ngrams(tokens, 2)`.
17. Count bigram frequencies using `Counter`.
18. Print the top 5 most common bigrams using `.most_common(5)`.

Expected Output:

```
Top 5 Bigrams:
('machine', 'learning') : 2
('learning', 'is')      : 2
('is', 'a')             : 2
('a', 'subset')         : 2
('subset', 'of')        : 2
```

```
# Step 15: Imports
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter
import string

# Ensure tokenizer is available
for resource in ['punkt', 'punkt_tab']:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource)

# Paragraph
paragraph = """Deep learning is a subset of machine learning.
Machine learning is a subset of artificial intelligence.
Deep learning uses neural networks."""

# Step 15: Tokenize and convert to lowercase
tokens = word_tokenize(paragraph.lower())

# Optional: Remove punctuation for cleaner bigrams
tokens = [w for w in tokens if w not in string.punctuation]

# Step 16: Generate bigrams
bigrams = list(ngrams(tokens, 2))

# Step 17: Count bigram frequencies
bigram_counts = Counter(bigrams)

# Step 18: Print top 5 most common bigrams
print("Top 5 Bigrams:")
for bigram, count in bigram_counts.most_common(5):
    print(f"{bigram}: {count}")

... Top 5 Bigrams:
('deep', 'learning'): 2
('learning', 'is'): 2
('is', 'a'): 2
('a', 'subset'): 2
('subset', 'of'): 2
```


Task 6: N-Grams on User-Provided Text

Write a program that takes a sentence from the user and a value of N, then displays all N-grams.

Steps:

19. Ask the user to enter a sentence.
20. Ask the user to enter the value of N.
21. Tokenize the sentence.
22. Generate N-grams using `ngrams(tokens, n)`.
23. Print all generated N-grams.

Example Interaction:

```
Enter a sentence: I love natural language processing
Enter value of N: 2
```

```
2-grams (Bigrams):
('i', 'love')
```

```
# Step 19-20: Imports
import nltk
from nltk.tokenize import word_tokenize
from nltk.util import ngrams

# Ensure tokenizer is available
for resource in ['punkt', 'punkt_tab']:
    try:
        nltk.data.find(resource)
    except LookupError:
        nltk.download(resource)

# Step 19: Take sentence input
sentence = input("Enter a sentence: ")

# Step 20: Take value of N
n = int(input("Enter value of N (e.g., 1, 2, 3): "))

# Step 21: Tokenize
tokens = word_tokenize(sentence.lower())

# Step 22: Generate N-grams
ngram_list = list(ngrams(tokens, n))

# Step 23: Print N-grams
print(f"\n{n}-grams:")
for gram in ngram_list:
    print(gram)
```

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
Enter a sentence: Machine learning is powerful
Enter value of N (e.g., 1, 2, 3): 3

3-grams:
('machine', 'learning', 'is')
('learning', 'is', 'powerful')
```

Task 7: Character N-Grams on a Word

Write a program that generates character-level bigrams and trigrams from a user-entered word.

Steps:

24. Ask the user to enter a word.
25. Convert the word to lowercase.
26. Generate character bigrams (each pair of consecutive characters).
27. Generate character trigrams (each trio of consecutive characters).
28. Print both results.

Example Interaction:

```
Enter a word: Python
```

```
Character Bigrams: ['py', 'yt', 'th', 'ho', 'on']
Character Trigrams: ['pyt', 'yth', 'tho', 'hon']
```

```
# Step 24: Take input from user
word = input("Enter a word: ")

# Step 25: Convert to lowercase
word = word.lower()

# Step 26: Generate character bigrams
bigrams = [word[i:i+2] for i in range(len(word) - 1)]

# Step 27: Generate character trigrams
trigrams = [word[i:i+3] for i in range(len(word) - 2)]

# Step 28: Print results
print("Character Bigrams:", bigrams)
print("Character Trigrams:", trigrams)
```

```
*** Enter a word: Learning
Character Bigrams: ['le', 'ea', 'ar', 'rn', 'ni', 'in', 'ng']
Character Trigrams: ['lea', 'ear', 'arn', 'rni', 'nin', 'ing']
```

Quick Comparison: BoW vs N-Grams

Feature	Bag of Words (BoW)	N-Grams
Definition	Counts individual word frequencies	Counts sequences of N consecutive words
Word Order	Ignored completely	Preserved within each n-gram window
Context	No context captured	Local context captured

Vocabulary Size	Equal to unique word count	Grows exponentially with N
Common N Value	N = 1 (unigrams)	N = 2 (bigrams), N = 3 (trigrams)

Feature	Bag of Words (BoW)	N-Grams
NLTK Function	Counter() + word_tokenize()	nltk.util.ngrams()
Use Cases	Document classification, spam detection	Language models, autocomplete, translation
Limitation	Loses meaning from word order	High N causes data sparsity

Lab # 07: TF-IDF Implementation

Lab Number	Lab # 07
Title	TF-IDF Implementation
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Understand and implement Term Frequency, Inverse Document Frequency, and TF-IDF vectorization manually and with Scikit-learn.

1. Theory

1.1 What is TF-IDF?

TF-IDF (Term Frequency – Inverse Document Frequency) is a numerical statistic used to reflect how important a word is to a document within a corpus. It is one of the most widely used techniques in information retrieval and text mining as a baseline feature extraction method before applying machine learning models.

1.2 Term Frequency (TF)

Term Frequency measures how often a term appears in a document, normalized by the total number of terms in that document. A high TF means the term appears frequently in the document.

Formula: $TF(t, d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of terms in } d)$

1.3 Inverse Document Frequency (IDF)

IDF measures how rare or common a term is across the entire corpus. Words that appear in many documents (e.g., 'the', 'is') are penalized; rare, informative words receive a higher IDF score.

Formula: $IDF(t, D) = \log(N / df(t))$

Where N is the total number of documents and $df(t)$ is the number of documents containing term t .

1.4 TF-IDF Score

Formula: $TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$

A high TF-IDF value means the term is frequent in a specific document but rare across the corpus — making it a strong descriptor of that document.

1.5 Manual Implementation

```
import math

corpus = [
    'the cat sat on the mat',
    'the dog sat on the log',
    'the cat chased the dog'
]

def compute_tf(doc):
    words = doc.split()
    tf = {}
    for w in words:
        tf[w] = tf.get(w, 0) + 1 / len(words)
    return tf

def compute_idf(corpus):
    N = len(corpus)
    idf = {}
    all_words = set(w for doc in corpus for w in doc.split())
    for word in all_words:
        df = sum(1 for doc in corpus if word in doc.split())
        idf[word] = math.log(N / df)
    return idf

def compute_tfidf(doc, idf):
    tf = compute_tf(doc)
    return {w: tf[w] * idf[w] for w in tf}

idf = compute_idf(corpus)
for i, doc in enumerate(corpus):
    scores = compute_tfidf(doc, idf)
    print(f'Doc {i+1}: {scores}')
```

1.6 TF-IDF with Scikit-learn

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print('Feature names:', vectorizer.get_feature_names_out())
print('TF-IDF Matrix:')
print(X.toarray())

```

1.7 Tkinter GUI Application

The following Tkinter application lets the user type multiple documents, then computes and displays TF-IDF scores:

```

import tkinter as tk
from tkinter import ttk, scrolledtext
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

def compute_tfidf():
    raw = input_box.get('1.0', tk.END).strip()
    docs = [d.strip() for d in raw.split('\n') if d.strip()]
    if len(docs) < 2:
        result_box.delete('1.0', tk.END)
        result_box.insert(tk.END, 'Please enter at least 2 documents.')
        return
    vec = TfidfVectorizer()
    X = vec.fit_transform(docs)
    df = pd.DataFrame(X.toarray(),
                      columns=vec.get_feature_names_out(),
                      index=[f'Doc {i+1}' for i in range(len(docs))])
    result_box.delete('1.0', tk.END)
    result_box.insert(tk.END, df.round(4).to_string())

root = tk.Tk()
root.title('TF-IDF Calculator')
root.geometry('700x500')

```

```

tk.Label(root, text='Enter one document per line:',
        font=('Arial',11,'bold')).pack(pady=5)
input_box = scrolledtext.ScrolledText(root, height=8, font=('Courier',10))
input_box.pack(fill=tk.X, padx=10)

tk.Button(root, text='Compute TF-IDF', command=compute_tfidf,
        bg='black', fg='white', font=('Arial',10,'bold')).pack(pady=8)

tk.Label(root, text='TF-IDF Matrix:', font=('Arial',11,'bold')).pack()
result_box = scrolledtext.ScrolledText(root, height=12, font=('Courier',9))
result_box.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)

root.mainloop()

```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Manually implement the TF computation function. Test it on a document of your choice and print the TF score for every word. Verify that all TF scores in a document sum to 1.0.	—
Task 2	Extend the manual TF-IDF implementation to a corpus of 5 short sentences. Print the TF-IDF score matrix neatly. Identify the word with the highest TF-IDF score in each document.	—
Task 3	Using TfidfVectorizer from sklearn, vectorize a corpus of 6 sentences. Print the feature names (vocabulary) and the shape of the resulting TF-IDF matrix.	—
Task 4	Modify the Tkinter TF-IDF application to also display the top 3 highest-scoring words for each document below the matrix table.	—
Task 5	Add a 'Clear' button to the Tkinter TF-IDF GUI that clears both the input and output text boxes. Also add an entry widget that allows the user to set the max_features parameter of TfidfVectorizer.	—

TASK 1

```

▶ import math

# Sample input text block
document = "the quick brown fox jumps over the lazy dog"

def compute_raw_tf(doc):
    words = doc.lower().split()
    total_word_count = len(words)
    tf_dict = {}

    for word in words:
        tf_dict[word] = tf_dict.get(word, 0) + 1

    # Normalize frequencies by total word length
    normalized_tf = {word: count / total_word_count for word, count in tf_dict.items()}
    return normalized_tf

# Calculate scores
tf_scores = compute_raw_tf(document)

print("--- Task 1: Manual TF Scores ---")
for word, score in tf_scores.items():
    print(f" '{word}': {score:.4f}")

# Verification verification step
total_sum = sum(tf_scores.values())
print(f"\nVerification Check: Sum of all TF elements = {total_sum:.1f}")

```

```

*** --- Task 1: Manual TF Scores ---
    'the': 0.2222
    'quick': 0.1111
    'brown': 0.1111
    'fox': 0.1111
    'jumps': 0.1111
    'over': 0.1111
    'lazy': 0.1111
    'dog': 0.1111

Verification Check: Sum of all TF elements = 1.0

```

TASK 2


```

import math
corpus = [
    'the cat sat on the mat',
    'the dog sat on the log',
    'the cat chased the dog',
    'the quick brown fox jumps',
    'lazy dogs sleep all day'
]

def get_tf(doc):
    words = doc.lower().split()
    return {w: words.count(w) / len(words) for w in words}

def get_idf(corpus_list):
    N = len(corpus_list)
    idf_dict = {}
    # Extract unique vocabulary across the entire corpus
    unique_words = set(w for doc in corpus_list for w in doc.lower().split())

    for word in unique_words:
        doc_count = sum(1 for doc in corpus_list if word in doc.lower().split())
        idf_dict[word] = math.log(N / doc_count)
    return idf_dict

corpus_idf = get_idf(corpus)

print("--- Task 2: Manual TF-IDF Matrix & Keywords ---")
for idx, doc in enumerate(corpus):
    doc_tf = get_tf(doc)
    tfidf_scores = {word: doc_tf[word] * corpus_idf[word] for word in doc_tf}

    # Identify the highest scoring token element
    highest_word = max(tfidf_scores, key=tfidf_scores.get)
    highest_score = tfidf_scores[highest_word]

    print(f"\nDocument {idx + 1}: \"{doc}\"")
    print(f" Top Key Token: '{highest_word}' with TF-IDF = {highest_score:.4f}")

... --- Task 2: Manual TF-IDF Matrix & Keywords ---

Document 1: "the cat sat on the mat"
Top Key Token: 'mat' with TF-IDF = 0.2682

Document 2: "the dog sat on the log"
Top Key Token: 'log' with TF-IDF = 0.2682

Document 3: "the cat chased the dog"
Top Key Token: 'chased' with TF-IDF = 0.3219

Document 4: "the quick brown fox jumps"
Top Key Token: 'quick' with TF-IDF = 0.3219

Document 5: "lazy dogs sleep all day"
Top Key Token: 'lazy' with TF-IDF = 0.3219

```

TASK 3

```

from sklearn.feature_extraction.text import TfidfVectorizer

six_sentence_corpus = [
    'Natural Language Processing is fun.',
    'Python is an amazing programming language.',
    'Machine learning models require data.',
    'TF-IDF measures word importance.',
    'Building GUI software with Tkinter.',
    'Scikit-learn makes machine learning easy.'
]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(six_sentence_corpus)

print("--- Task 3: Scikit-Learn Verification ---")
print("Extracted Vocabulary Feature Names:")
print(vectorizer.get_feature_names_out())
print(f"\nTF-IDF Matrix Shape Geometry Dimensions: {tfidf_matrix.shape}")

--- --- Task 3: Scikit-Learn Verification ---
Extracted Vocabulary Feature Names:
['amazing' 'an' 'building' 'data' 'easy' 'fun' 'gui' 'idf' 'importance'
 'is' 'language' 'learn' 'learning' 'machine' 'makes' 'measures' 'models'
 'natural' 'processing' 'programming' 'python' 'require' 'scikit'
 'software' 'tf' 'tkinter' 'with' 'word']

TF-IDF Matrix Shape Geometry Dimensions: (6, 28)

```

TASK 4&5

```

import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

def process_tfidf_pipeline():
    raw_input = text_input.get('1.0', tk.END).strip()
    documents = [line.strip() for line in raw_input.split('\n') if line.strip()]

    if len(documents) < 2:
        messagebox.showwarning("Data Limit Error", "Please input at least 2 documentation
lines.")
        return

    # Task 5: Handle dynamic entry constraint variables safely
    max_features_val = max_feat_entry.get().strip()
    parsed_max = int(max_features_val) if max_features_val.isdigit() else None

```

try:

```

vec = TfidfVectorizer(max_features=parsed_max)
matrix_sparse = vec.fit_transform(documents)
feature_vocab = vec.get_feature_names_out()

# Build DataFrame for visualization structured rows
df = pd.DataFrame(matrix_sparse.toarray(),
                  columns=feature_vocab,
                  index=[f'Doc_{i+1}' for i in range(len(documents))])

display_box.delete('1.0', tk.END)
display_box.insert(tk.END, "=== TF-IDF Spark Matrix Frame ===\n")
display_box.insert(tk.END, df.round(4).to_string() + "\n\n")

# Task 4: Dynamic analytical scanning loop for top 3 terms per document row
display_box.insert(tk.END, "=== Top 3 Informative Keywords Per Document Row\n")
for index_lbl, row_series in df.iterrows():
    top_3_series = row_series.nlargest(3)
    keyword_strings = [f'{word} ({score:.4f})' for word, score in top_3_series.items() if
score > 0]
    display_box.insert(tk.END, f'{index_lbl}: {' .join(keyword_strings) if keyword_strings
else 'None'}\n')

except Exception as err:
    messagebox.showerror("Pipeline Failure", f"Processing runtime exception occurred:
{str(err)}")

# Task 5: Clear UI function
def reset_workspace_fields():
    text_input.delete('1.0', tk.END)
    display_box.delete('1.0', tk.END)
    max_feat_entry.delete(0, tk.END)

# GUI Initialization Layout Components
root = tk.Tk()

```

```
root.title("Advanced Automated TF-IDF Analytical Workstation")
root.geometry("750x600")

tk.Label(root, text="Enter Text Streams (One Document Per Row Line):", font=('Arial', 10,
'bold')).pack(pady=5)
text_input = scrolledtext.ScrolledText(root, height=8, font=('Courier', 10))
text_input.pack(fill=tk.X, padx=15)

# Task 5: Configuration parameters bar panel structure
config_panel = tk.Frame(root)
config_panel.pack(pady=5)
tk.Label(config_panel, text="Max Feature Constrain Limits (Optional):", font=('Arial',
9)).pack(side=tk.LEFT, padx=5)
max_feat_entry = tk.Entry(config_panel, width=10)
max_feat_entry.pack(side=tk.LEFT, padx=5)

control_panel = tk.Frame(root)
control_panel.pack(pady=5)
tk.Button(control_panel, text="Compute TF-IDF Matrix", command=process_tfidf_pipeline,
bg="black", fg="white", font=('Arial', 10, 'bold')).pack(side=tk.LEFT, padx=5)
tk.Button(control_panel, text="Reset Fields", command=reset_workspace_fields,
bg="darkgray", fg="black").pack(side=tk.LEFT, padx=5)

tk.Label(root, text="Analytical Breakdown Console Frames:", font=('Arial', 10,
'bold')).pack(pady=5)
display_box = scrolledtext.ScrolledText(root, height=16, font=('Courier', 9))
display_box.pack(fill=tk.BOTH, expand=True, padx=15, pady=10)

root.mainloop()
```

Lab # 08: Sentiment Analysis

Lab Number	Lab # 08
Title	Sentiment Analysis
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Understand sentiment analysis concepts; implement rule-based positive/negative text classification using VADER and NLTK.

1. Theory

1.1 What is Sentiment Analysis?

Sentiment Analysis (also called Opinion Mining) is the use of NLP to identify and extract subjective information from text — primarily whether the expressed opinion is positive, negative, or neutral. It is widely used in product reviews, social media monitoring, customer feedback systems, and financial news analysis.

1.2 Levels of Sentiment Analysis

- Document Level: Classifies the overall sentiment of an entire document.
- Sentence Level: Classifies each sentence individually.
- Aspect Level: Identifies sentiment toward specific aspects (e.g., 'The battery life is great but the camera is poor').

1.3 Approaches

Rule-Based (Lexicon-Based): Uses a dictionary of words pre-labeled with sentiment scores. No machine learning is involved. Fast and interpretable but limited to known vocabulary.

Machine Learning: Trains a classifier on labeled sentiment data. More flexible but requires annotated training data.

Deep Learning: Uses RNNs, LSTMs, or Transformers (BERT) for context-aware sentiment understanding.

1.4 VADER Sentiment Analyzer

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based sentiment

Natural language processing lab manual
analysis tool in NLTK specifically designed for social media text. It handles emoticons, slang, capitalization, and punctuation. It returns four scores:

- pos – proportion of positive sentiment
- neg – proportion of negative sentiment
- neu – proportion of neutral sentiment
- compound – normalized overall score between -1 (most negative) and +1 (most positive)

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
nltk.download('vader_lexicon')
```

```
sia = SentimentIntensityAnalyzer()
```

```
texts = [
    'I absolutely love this product! It is amazing.',
    'This is the worst experience I have ever had.',
    'The item arrived on time.'
]
```

```
for text in texts:
    scores = sia.polarity_scores(text)
    label = 'Positive' if scores['compound'] >= 0.05 else \
        'Negative' if scores['compound'] <= -0.05 else 'Neutral'
    print(f'{label:9s} | {scores} | {text[:40]}')
```

1.5 Simple Lexicon-Based Classifier

```
positive_words = {'good', 'great', 'excellent', 'love', 'amazing', 'happy', 'best'}
negative_words = {'bad', 'terrible', 'awful', 'hate', 'worst', 'poor', 'sad'}
```

```
def classify(text):
    words = text.lower().split()
    pos = sum(1 for w in words if w in positive_words)
    neg = sum(1 for w in words if w in negative_words)
    if pos > neg: return 'Positive'
    if neg > pos: return 'Negative'
    return 'Neutral'
```

1.6 Tkinter Sentiment Analysis App

```

import tkinter as tk
from tkinter import scrolledtext
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
nltk.download('vader_lexicon', quiet=True)

sia = SentimentIntensityAnalyzer()

def analyze():
    text = entry.get('1.0', tk.END).strip()
    scores = sia.polarity_scores(text)
    comp = scores['compound']
    if comp >= 0.05:
        label, color = 'POSITIVE', 'green'
    elif comp <= -0.05:
        label, color = 'NEGATIVE', 'red'
    else:
        label, color = 'NEUTRAL', 'gray'
    result_label.config(text=f'Sentiment: {label} (compound={comp:.3f})',
                        fg=color)
    detail.config(state='normal')
    detail.delete('1.0', tk.END)
    detail.insert(tk.END,
        f'Positive : {scores["pos"]:.3f}\n'
        f'Negative : {scores["neg"]:.3f}\n'
        f'Neutral : {scores["neu"]:.3f}\n'
        f'Compound : {comp:.3f}')
    detail.config(state='disabled')

root = tk.Tk()
root.title('Sentiment Analyzer')
root.geometry('550x400')

tk.Label(root, text='Sentiment Analysis Tool',
        font=('Arial', 14, 'bold')).pack(pady=8)

```

```

tk.Label(root, text='Enter text below:',
        font=('Arial',10)).pack()
entry = scrolledtext.ScrolledText(root, height=6, font=('Arial',10))
entry.pack(fill=tk.X, padx=15, pady=5)
tk.Button(root, text='Analyze Sentiment', command=analyze,
        bg='black', fg='white', font=('Arial',10,'bold')).pack(pady=6)
result_label = tk.Label(root, text="", font=('Arial',13,'bold'))
result_label.pack()
detail = scrolledtext.ScrolledText(root, height=5, font=('Courier',10),
        state='disabled')
detail.pack(fill=tk.X, padx=15, pady=5)
root.mainloop()

```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Write a Python program (without GUI) that reads a list of 10 product reviews and uses VADER to classify each as Positive, Negative, or Neutral. Print the review, compound score, and label for each.	—
Task 2	Build a custom lexicon-based classifier using two sets of at least 15 positive and 15 negative words. Test it on 8 sentences and compare its results with VADER's output.	—
Task 3	Extend the Tkinter Sentiment App to accept multiple sentences (one per line) and display a summary: how many sentences were Positive, Negative, and Neutral.	—
Task 4	Add a progress bar or percentage bar to the Tkinter app that visually shows the compound score (from -1 to +1) as a horizontal bar using tkinter.ttk.Progressbar.	—
Task 5	Create a Tkinter app with a Listbox showing a history of the last 5 analyzed texts along with their sentiment labels. Each new analysis should be prepended to the history list.	—

Task 1


```

import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
nltk.download('vader_lexicon', quiet=True)

reviews_list = [
    "This laptop is lightning fast and has a beautiful screen!",
    "Terrible customer service. It arrived broken and they refused a refund.",
    "The book is okay, nothing exceptional but worth a quick read.",
    "I absolutely love the camera quality on this phone.",
    "Worst experience ever. The app crashes every single time I open it.",
    "It works fine, pretty standard performance for the price tag.",
    "The food was delicious, but the seating arrangement was cramped.",
    "Extremely disappointed with the cheap material used.",
    "Highly recommended product, sleek design and functional.",
    "It is an average movie, has some good spots and some slow parts."
]

vader_analyzer = SentimentIntensityAnalyzer()

print(f"{'PREDICTION LBL':15s} | {'COMPOUND METRIC':15s} | {'RAW STRING FRAGMENT'}")
print("-" * 75)
for text_review in reviews_list:
    score_map = vader_analyzer.polarity_scores(text_review)
    comp = score_map['compound']

    label = "Positive" if comp >= 0.05 else "Negative" if comp <= -0.05 else "Neutral"
    print(f"{label:15s} | {comp:15.4f} | {text_review[:45]}...")

```

```

... PREDICTION LBL | COMPOUND METRIC | RAW STRING FRAGMENT
-----
Positive | 0.6360 | This laptop is lightning fast and has a beaut...
Negative | -0.8126 | Terrible customer service. It arrived broken ...
Negative | -0.1403 | The book is okay, nothing exceptional but wor...
Positive | 0.6697 | I absolutely love the camera quality on this ...
Negative | -0.6249 | Worst experience ever. The app crashes every ...
Positive | 0.6124 | It works fine, pretty standard performance fo...
Positive | 0.3291 | The food was delicious, but the seating arran...
Negative | -0.5256 | Extremely disappointed with the cheap materia...
Positive | 0.2716 | Highly recommended product, sleek design and ...
Positive | 0.4404 | It is an average movie, has some good spots a...

```

Task 2

```

from nltk.sentiment.vader import SentimentIntensityAnalyzer

pos_keywords = {'love', 'beautiful', 'perfect', 'fantastic', 'great', 'nice', 'stable'}
neg_keywords = {'terrible', 'waste', 'fail', 'disappointed', 'broken', 'horrible', 'bad'}

def custom_lexicon_classifier(text_string):
    tokens = text_string.lower().split()
    pos_count = sum(1 for token in tokens if token in pos_keywords)
    neg_count = sum(1 for token in tokens if token in neg_keywords)

    if pos_count > neg_count: return "Positive"
    if neg_count > pos_count: return "Negative"
    return "Neutral"

test_sentences = [
    "I love this beautiful and perfect day.",
    "This is a terrible waste of time and a total fail.",
    "The engine works fine and is quite stable.",
    "I am extremely disappointed with the broken device.",
    "The views were fantastic and we had a great time.",
    "It was a bad movie but it had some nice music.",
    "The shipment arrived on time.",
    "I hate getting stuck in horrible traffic."
]

vader = SentimentIntensityAnalyzer()

print(f"{'TARGET STRING SENTENCE':48s} | {'CUSTOM LAB':10s} | {'VADER LAB'}")
print("-" * 75)
for sent in test_sentences:
    custom_res = custom_lexicon_classifier(sent)
    vader_score = vader.polarity_scores(sent)['compound']
    vader_res = "Positive" if vader_score >= 0.05 else "Negative" if vader_score <= -0.05 else "Neutral"
    print(f"{sent[:45]:48s} | {custom_res:10s} | {vader_res}")

```

TARGET STRING SENTENCE	CUSTOM LAB	VADER LAB
I love this beautiful and perfect day.	Positive	Positive
This is a terrible waste of time and a total fail.	Negative	Negative
The engine works fine and is quite stable.	Neutral	Positive
I am extremely disappointed with the broken d	Negative	Negative
The views were fantastic and we had a great t	Positive	Positive
It was a bad movie but it had some nice music	Neutral	Positive
The shipment arrived on time.	Neutral	Neutral
I hate getting stuck in horrible traffic.	Negative	Negative

Task 3,4,5

import tkinter as tk

from tkinter import scrolledtext, ttk

from nltk.sentiment.vader import SentimentIntensityAnalyzer

```
vader_unit = SentimentIntensityAnalyzer()
```

```
historical_runs_tracker = []
```

```
def perform_sentiment_pipeline_analysis():
```

```
    raw_text_block = text_box.get('1.0', tk.END).strip()
```

```
    if not raw_text_block: return
```

```
    lines = [line.strip() for line in raw_text_block.split('\n') if line.strip()]
```

```
    pos_lines, neg_lines, neu_lines = 0, 0, 0
```

```
    aggregate_compound_score = 0.0
```

```
    # Task 3: Processing line-by-line iterations safely
```

```
    for individual_line in lines:
```

```
        scores = vader_unit.polarity_scores(individual_line)
```

```
        comp_val = scores['compound']
```

```
        aggregate_compound_score += comp_val
```

```
        if comp_val >= 0.05: pos_lines += 1
```

```
        elif comp_val <= -0.05: neg_lines += 1
```

```
        else: neu_lines += 1
```

```
    mean_compound_score = aggregate_compound_score / len(lines) if lines else 0.0
```

```
    if mean_compound_score >= 0.05: final_verdict, metric_color = "POSITIVE", "green"
```

```
    elif mean_compound_score <= -0.05: final_verdict, metric_color = "NEGATIVE", "red"
```

```
    else: final_verdict, metric_color = "NEUTRAL", "dimgray"
```

```
    summary_verdict_lbl.config(text=f"Summary Verdict: {final_verdict} (Mean Score:  
{mean_compound_score:.4f})", fg=metric_color)
```

```

# Update numerical stat tracking fields
stat_display_box.config(state='normal')
stat_display_box.delete('1.0', tk.END)
stat_display_box.insert(tk.END, f'Total Document Rows Parsed: {len(lines)}\n')
stat_display_box.insert(tk.END, f' Positive Row Count: {pos_lines}\n')
stat_display_box.insert(tk.END, f' Negative Row Count: {neg_lines}\n')
stat_display_box.insert(tk.END, f' Neutral Row Count : {neu_lines}\n')
stat_display_box.config(state='disabled')

# Task 4: Map compound metrics bounds (-1 to +1) to progressbar scales (0 to 100)
transformed_percentage = ((mean_compound_score - (-1.0)) / 2.0) * 100.0
visual_progress_bar['value'] = transformed_percentage

# Task 5: Prepend logging string structures to rolling trace display frame tracking limit constraints
log_summary_string = f"[{final_verdict[:3]}] {raw_text_block[:30].strip()}..."
historical_runs_tracker.insert(0, log_summary_string)

if len(historical_runs_tracker) > 5:
    historical_runs_tracker.pop()

history_list_component.delete(0, tk.END)
for index_pos, log_item in enumerate(historical_runs_tracker):
    history_list_component.insert(index_pos, log_item)

root = tk.Tk()
root.title("Enterprise Sentiment Analytics Suite Engine")
root.geometry("620(650)")
root.geometry("620x650")

tk.Label(root, text="Enter Sentences Block Area for Metrics Processing:", font=('Arial', 10,
'bold')).pack(pady=5)

```

```
text_box = scrolledtext.ScrolledText(root, height=6, font=('Arial', 10))
```

```
text_box.pack(fill=tk.X, padx=15)
```

```
tk.Button(root, text="Execute Sentiment Evaluation", command=perform_sentiment_pipeline_analysis,  
bg="black", fg="white", font=('Arial', 10, 'bold')).pack(pady=8)
```

```
summary_verdict_lbl = tk.Label(root, text="Summary Verdict: Pending Run", font=('Arial', 11, 'bold'))
```

```
summary_verdict_lbl.pack()
```

```
# Task 4: Visualizing compound metrics scale configuration component
```

```
tk.Label(root, text="Compound Valence Scale Balance Position Indicator (-1 to +1 Boundary Map):",  
font=('Arial', 9, 'italic')).pack(pady=2)
```

```
visual_progress_bar = ttk.Progressbar(root, orient='horizontal', length=450, mode='determinate')
```

```
visual_progress_bar.pack(pady=5)
```

```
stat_display_box = scrolledtext.ScrolledText(root, height=5, font=('Courier', 10), state='disabled')
```

```
stat_display_box.pack(fill=tk.X, padx=15, pady=5)
```

```
# Task 5: History trace frame widget architecture panel
```

```
tk.Label(root, text="Rolling Transaction Run History Dashboard Tracker (Max 5 Trace Logs):", font=('Arial',  
10, 'bold')).pack(pady=5)
```

```
history_list_component = tk.Listbox(root, height=6, font=('Courier', 10))
```

```
history_list_component.pack(fill=tk.BOTH, expand=True, padx=15, pady=10)
```

```
root.mainloop()
```

Lab # 09: Text Classification using Machine Learning

Lab Number	Lab # 09
Title	Text Classification using Machine Learning
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Prepare a text dataset, perform train-test split, and train a Naive Bayes classifier for text classification.

1. Theory

1.1 Text Classification

Text classification assigns predefined categories to text documents. Applications include spam detection, topic labeling, language detection, news categorization, and intent detection in chatbots. The ML pipeline for text classification typically consists of: raw text → preprocessing → feature extraction → model training → prediction.

1.2 Dataset Preparation

Before training, text data must be prepared:

- Lowercasing: Convert all text to lowercase.
- Tokenization: Split text into individual words (tokens).
- Stop Word Removal: Remove common words like 'the', 'is', 'and' that carry little meaning.
- Punctuation Removal: Strip non-alphabetic characters.
- Stemming / Lemmatization: Reduce words to their root form.

```
import re
from nltk.corpus import stopwords
import nltk
nltk.download('stopwords', quiet=True)

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
```

```

text = re.sub(r'^a-z\s', '', text)
tokens = text.split()
tokens = [t for t in tokens if t not in stop_words]
return ' '.join(tokens)

```

1.3 Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' Theorem with the 'naive' assumption that all features (words) are conditionally independent given the class label. Despite this simplification, it performs surprisingly well on text classification tasks and is extremely fast to train.

Bayes' Theorem: $P(\text{class} \mid \text{words})$ proportional to $P(\text{words} \mid \text{class}) \times P(\text{class})$

For text: the probability of a document belonging to a class is proportional to the product of the probabilities of each word appearing in that class, multiplied by the prior probability of the class.

MultinomialNB: Best suited for word count or TF-IDF features. ComplementNB: Improved variant, better for imbalanced datasets.

1.4 Full Implementation

```

from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Use a subset of categories for speed
categories = ['sci.space', 'rec.sport.hockey', 'talk.politics.guns', 'comp.graphics']
data = fetch_20newsgroups(subset='all', categories=categories)

X_raw = [preprocess(t) for t in data.data]
y = data.target

X_train_r, X_test_r, y_train, y_test = train_test_split(
    X_raw, y, test_size=0.2, random_state=42)

```

```

vec = TfidfVectorizer(max_features=5000)
X_train = vec.fit_transform(X_train_r)
X_test = vec.transform(X_test_r)

model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred, target_names=data.target_names))

```

1.5 Tkinter Text Classifier App

```

import tkinter as tk
from tkinter import scrolledtext, messagebox
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

# Training data (spam vs ham)
train_texts = [
    'Win a free iPhone now click here',
    'Congratulations you have won a prize',
    'Limited offer buy now discount',
    'Hello how are you doing today',
    'Meeting scheduled for tomorrow morning',
    'Please review the attached document',
]
train_labels = ['spam', 'spam', 'spam', 'ham', 'ham', 'ham']

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('nb', MultinomialNB())
])
pipeline.fit(train_texts, train_labels)

def classify_text():
    text = text_box.get('1.0', tk.END).strip()

```



```

if not text:
    messagebox.showwarning('Input', 'Please enter some text.')
    return
pred = pipeline.predict([text])[0]
prob = pipeline.predict_proba([text])[0]
result_var.set(f'Prediction: {pred.upper()}')
conf_var.set(f'Confidence – Spam: {prob[1]:.2f} Ham: {prob[0]:.2f}')

root = tk.Tk()
root.title('Spam / Ham Classifier')
root.geometry('500x360')
tk.Label(root, text='Text Classifier (Spam Detector)',
         font=('Arial',13,'bold')).pack(pady=8)
text_box = scrolledtext.ScrolledText(root, height=7, font=('Arial',10))
text_box.pack(fill=tk.X, padx=15)
tk.Button(root, text='Classify', command=classify_text,
         bg='black', fg='white', font=('Arial',10,'bold')).pack(pady=8)
result_var = tk.StringVar()
conf_var = tk.StringVar()
tk.Label(root, textvariable=result_var, font=('Arial',13,'bold')).pack()
tk.Label(root, textvariable=conf_var, font=('Arial',10)).pack()
root.mainloop()

```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Create a small labeled dataset of 20 text samples with two categories of your choice (e.g., sports vs technology). Apply the preprocessing function (lowercase, remove stop words, remove punctuation) to all samples. Print before and after for 3 samples.	—
Task 2	Using the preprocessed dataset from Task 1, vectorize with TfidfVectorizer and train a MultinomialNB classifier. Perform an 80/20 train-test split. Print the accuracy score.	—
Task	Modify the Tkinter classifier app to allow the user to choose	—

3	between two categories (e.g., Positive/Negative reviews). Add at least 8 training samples per category. Display the prediction and confidence scores.	
Task 4	Add a 'Training Data' window to the Tkinter app (opened via a button) that shows the training texts and their labels in a Listbox widget.	—
Task 5	Extend the classifier to use three categories (e.g., Sports, Technology, Politics). Update the training data and the GUI to handle multi-class classification. Display all class probabilities.	—

Task 1&2

```

import re
import nltk
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

nltk.download('stopwords', quiet=True)
stop_words_set = set(stopwords.words('english'))

# Task 1: Complete 20-element customized structured source text corpus list array
custom_domain_corpus = [
    ("The striker scored a spectacular hat-trick during the match", "sports"),
    ("Basketball teams must practice defensive positioning drills daily", "sports"),
    ("The championship games were delayed due to lightning warnings", "sports"),
    ("Olympic athletes cycle through grueling conditioning regimens every season", "sports"),
    ("The tennis tournament organizer finalized brackets for men's doubles", "sports"),
    ("A gold medal was awarded after the record-breaking sprint run", "sports"),
    ("Midfielders navigate structural spaces to pass footballs efficiently", "sports"),
    ("The gymnastics coach commended her perfect balance beam routine", "sports"),
    ("The referee issued three yellow cards for aggressive behavior", "sports"),
    ("Swimming relay squads practice synchronized diving exchanges consistently", "sports"),
    ("Our system backend runs containerized microservices across cloud frameworks", "technology"),
    ("This secure database cluster handles sharded structured query operations", "technology"),
    ("Software engineers push clean object-oriented Python source modules", "technology"),
    ("Advanced deep learning systems require localized GPU server clusters", "technology"),
    ("Cybersecurity experts patch architectural kernel level memory leak exploits", "technology"),
    ("Git repository branches facilitate collaborative software distribution pipelines", "technology"),
    ("A cryptographic hashing algorithm secures distributed ledger transactional records", "technology"),
    ("Network routers utilize interface protocols to direct server traffic", "technology"),
    ("Frontend teams compile web application views with scalable frameworks", "technology"),
    ("Quantum computer hardware processes complex mathematical simulations quickly", "technology")
]

def standard_cleaning_pipeline(text_in):
    cleaned_lower = text_in.lower()
    stripped_string = re.sub(r'[^a-z\s]', '', cleaned_lower)
    tokens = [tok for tok in stripped_string.split() if tok not in stop_words_set]
    return ' '.join(tokens)

# Run verification logs for preprocessor checks
print("--- Task 1: Pipeline Preprocessing Trace Validation ---")
for index in range(2):
    raw_text, label_tag = custom_domain_corpus[index]
    print(f"[{label_tag.upper()}] Raw: {raw_text}")
    print(f"[{label_tag.upper()}] Cleaned: {standard_cleaning_pipeline(raw_text)}\n")

# Task 2: Implement Train Test splits and evaluate execution outcomes
processed_texts = [standard_cleaning_pipeline(item[0]) for item in custom_domain_corpus]
target_labels = [item[1] for item in custom_domain_corpus]
X_train, X_test, y_train, y_test = train_test_split(processed_texts, target_labels, test_size=0.20, random_state=101)
tfidf_transformer = TfidfVectorizer()
X_train_transformed = tfidf_transformer.fit_transform(X_train)
X_test_transformed = tfidf_transformer.transform(X_test)
naive_bayes_model = MultinomialNB()
naive_bayes_model.fit(X_train_transformed, y_train)
predictions = naive_bayes_model.predict(X_test_transformed)
final_accuracy = accuracy_score(y_test, predictions)
print("--- Task 2: Classifier Test Outcome Evaluation ---")
print(f"Evaluated General Matrix Accuracy Score: {final_accuracy * 100:.2f}%")

*** --- Task 1: Pipeline Preprocessing Trace Validation ---
[SPORTS] Raw: The striker scored a spectacular hat-trick during the match
[SPORTS] Cleaned: striker scored spectacular hattrick match

[SPORTS] Raw: Basketball teams must practice defensive positioning drills daily
[SPORTS] Cleaned: basketball teams must practice defensive positioning drills daily

--- Task 2: Classifier Test Outcome Evaluation ---
Evaluated General Matrix Accuracy Score: 0.00%

```

Task 3,4,5

```

import tkinter as tk

from tkinter import scrolledtext, messagebox

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.pipeline import Pipeline

# Task 3 & 5: Explicit multi-class text corpus dictionary data arrays
training_texts_matrix = [
    'Outstanding pristine excellent elite perfect favorite highly recommended', 'positive',
    'Amazing gorgeous wonderful stellar functionality brilliant clean super', 'positive',
    'Terrible defect broken garbage absolute waste horizontal failure ruined', 'negative',
    'Horrible stop functioning return refund customer service nightmare awful', 'negative',
    'Standard basic generic okay fair expected normal typical intermediate', 'neutral',
    'Mediocre passing regular ordinary plain unexciting functional baseline', 'neutral'
]

training_labels_matrix = ['positive', 'positive', 'negative', 'negative', 'neutral', 'neutral']

# Initialize unified automated pipeline structural context object
text_classifier_pipeline = Pipeline([
    ('vectorizer', TfidfVectorizer()),
    ('classifier', MultinomialNB())
])

text_classifier_pipeline.fit(training_texts_matrix, training_labels_matrix)

def evaluate_target_input_string():
    input_text = text_entry_field.get('1.0', tk.END).strip()
    if not input_text:
        messagebox.showwarning("Validation Error Alert", "Please insert a text snippet to continue.")
    return

```

```

predicted_target_class = text_classifier_pipeline.predict([input_text])[0]
prob_distribution_weights = text_classifier_pipeline.predict_proba([input_text])[0]
class_labels_mapping = text_classifier_pipeline.classes_

verdict_string_var.set(f'Predicted Class Verdict Target: {predicted_target_class.upper()}')

# Task 5: Dynamic class metric distribution loop generation
distribution_log_string = " | ".join([f'{lbl.capitalize()}: {weight:.3f}' for lbl, weight in
zip(class_labels_mapping, prob_distribution_weights)])
prob_string_var.set(distribution_log_string)

# Task 4: Sub-window frame generation for database inspection features
def render_training_source_subwindow():
    viewer_subwindow = tk.Toplevel(root)
    viewer_subwindow.title("Internal Source Training Repository Database Viewer")
    viewer_subwindow.geometry("520x280")

    tk.Label(viewer_subwindow, text="Active Pipeline Training Document Records Matrix", font=('Arial', 10,
'bold')).pack(pady=5)

    records_display_list = tk.Listbox(viewer_subwindow, font=('Courier', 9), width=70)
    records_display_list.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    for string_doc, label_tag in zip(training_texts_matrix, training_labels_matrix):
        records_display_list.insert(tk.END, f' [{label_tag.upper():8s}] -> {string_doc}')

root = tk.Tk()
root.title("Multi-Class Probabilistic Sentiment Classification Engine")
root.geometry("560x420")

tk.Label(root, text="Insert Custom Critique or Review Segment for Categorization:", font=('Arial', 10,
'bold')).pack(pady=5)
text_entry_field = scrolledtext.ScrolledText(root, height=6, font=('Arial', 10))

```

```
text_entry_field.pack(fill=tk.X, padx=15)
```

```
button_group_panel = tk.Frame(root)
```

```
button_group_panel.pack(pady=8)
```

```
tk.Button(button_group_panel, text="Run Classification Pipeline", command=evaluate_target_input_string,  
bg="black", fg="white", font=('Arial', 10, 'bold')).pack(side=tk.LEFT, padx=5)
```

```
tk.Button(button_group_panel, text="Inspect Train Data Store",  
command=render_training_source_subwindow, bg="dimgray", fg="white").pack(side=tk.LEFT, padx=5)
```

```
verdict_string_var = tk.StringVar(value="Predicted Class Verdict Target: Waiting Input")
```

```
prob_string_var = tk.StringVar()
```

```
tk.Label(root, textvariable=verdict_string_var, font=('Arial', 11, 'bold'), fg="darkblue").pack(pady=5)
```

```
tk.Label(root, textvariable=prob_string_var, font=('Courier', 10, 'bold'), fg="purple").pack(pady=5)
```

```
root.mainloop()
```

Lab # 10: NLP Pipeline using Scikit-learn

Lab Number	Lab # 10
Title	NLP Pipeline using Scikit-learn
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Build a complete NLP pipeline combining a vectorizer and classifier; evaluate model accuracy.

1. Theory

1.1 What is a Pipeline?

In Scikit-learn, a Pipeline chains multiple processing steps into a single object. It ensures that the same preprocessing transformations are applied consistently during both training and testing, preventing data leakage and reducing code repetition. For NLP, a typical pipeline is: text input → vectorizer → classifier → prediction.

1.2 Building an NLP Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

# Pipeline with CountVectorizer + TF-IDF + Classifier
pipeline = Pipeline([
    ('vect', CountVectorizer(stop_words='english')),
    ('tfidf', TfidfTransformer()),
    ('clf', MultinomialNB())
])
```

1.3 Training and Evaluating the Pipeline

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

```
categories = ['sci.med','comp.graphics','rec.sport.baseball','talk.religion.misc']
dataset = fetch_20newsgroups(subset='all', categories=categories)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    dataset.data, dataset.target, test_size=0.25, random_state=0)
```

```
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

```
print(f'Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%')
print(classification_report(y_test, y_pred,
    target_names=dataset.target_names))
```

1.4 Comparing Multiple Pipelines

```
classifiers = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Reg': LogisticRegression(max_iter=500),
    'Linear SVC': LinearSVC()
}

for name, clf in classifiers.items():
    pipe = Pipeline([('vect', CountVectorizer(stop_words='english')),
        ('tfidf', TfidfTransformer()),
        ('clf', clf)])
    pipe.fit(X_train, y_train)
    acc = accuracy_score(y_test, pipe.predict(X_test))
    print(f'{name:15s}: {acc*100:.2f}%')
```

1.5 Tkinter NLP Pipeline App

```
import tkinter as tk
from tkinter import scrolledtext, ttk
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```



```

# Sample training data – topic classification
train_texts = [
    'football goal match player team league',
    'basketball slam dunk court referee',
    'python code function algorithm program',
    'neural network deep learning training epoch',
    'medicine hospital doctor patient treatment',
    'surgery vaccine disease cure health',
    'election vote parliament government policy',
    'president prime minister cabinet law',
]
train_labels = ['Sports','Sports','Tech','Tech','Medical','Medical','Politics','Politics']

pipe = Pipeline([('tfidf', TfidfVectorizer()), ('clf', LogisticRegression())])
pipe.fit(train_texts, train_labels)

def run_pipeline():
    text = inp.get('1.0', tk.END).strip()
    if not text: return
    pred = pipe.predict([text])[0]
    probs = pipe.predict_proba([text])[0]
    classes = pipe.classes_
    out.config(state='normal')
    out.delete('1.0', tk.END)
    out.insert(tk.END, f'Predicted Category: {pred}\n\n')
    out.insert(tk.END, 'Class Probabilities:\n')
    for c, prob in zip(classes, probs):
        bar = '#' * int(prob * 30)
        out.insert(tk.END, f' {c:10s}: {bar} {prob:.3f}\n')
    out.config(state='disabled')

root = tk.Tk()
root.title('NLP Pipeline - Topic Classifier')
root.geometry('580x420')
tk.Label(root, text='NLP Pipeline Topic Classifier',
        font=('Arial',13,'bold')).pack(pady=8)
inp = scrolledtext.ScrolledText(root, height=7, font=('Arial',10))

```

```
inp.pack(fill=tk.X, padx=15)
tk.Button(root, text='Run Pipeline', command=run_pipeline,
          bg='black', fg='white', font=('Arial',10,'bold')).pack(pady=6)
out = scrolledtext.ScrolledText(root, height=9, font=('Courier',10),
                                state='disabled')
out.pack(fill=tk.BOTH, expand=True, padx=15, pady=5)
root.mainloop()
```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Build a Pipeline using CountVectorizer and MultinomialNB on any text classification dataset of your choice. Print the accuracy score and a classification report.	—
Task 2	Extend the pipeline to include TfidfTransformer between the vectorizer and classifier. Compare accuracy with and without TF-IDF weighting. Print both results.	—
Task 3	Evaluate three different classifiers (Naive Bayes, Logistic Regression, Linear SVC) in separate pipelines on the same dataset. Print a comparison table of their accuracy scores.	—
Task 4	Modify the Tkinter pipeline app to allow the user to select the classifier from a dropdown menu (Combobox) showing: 'Logistic Regression', 'Naive Bayes', 'Linear SVC'. Re-train and predict when the classifier is changed.	—
Task 5	Add a 'Prediction History' section to the Tkinter app (using a Listbox) that stores the last 8 predictions showing the input text (truncated to 30 chars) and predicted category.	—

Task 1,2,3

```

from sklearn.metrics import accuracy_score, classification_report
import pandas as pd
target_topics = ['sci.electronics', 'rec.autos', 'alt.atheism', 'sci.med']
newsgroups_source_data = fetch_20newsgroups(subset='all', categories=target_topics, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(newsgroups_source_data.data, newsgroups_source_data.target, test_size=0.25, random_state=42)
# Task 1: CountVectorizer Baseline Sequence Chaining Structure Loop
baseline_pipeline = Pipeline([
    ('count_vectorizer', CountVectorizer(stop_words='english')),
    ('naive_bayes_model', MultinomialNB())
])
baseline_pipeline.fit(X_train, y_train)
baseline_predictions = baseline_pipeline.predict(X_test)
print("=== Task 1: Baseline Pipeline Performance Metrics ===")
print(f"Accuracy: {accuracy_score(y_test, baseline_predictions)*100:.2f}%")
print(classification_report(y_test, baseline_predictions, target_names=newsgroups_source_data.target_names))
# Task 2: Impact Analysis of Scaling Adjustments (With vs Without TF-IDF Transformer Scaling)
scaled_pipeline = Pipeline([
    ('count_vectorizer', CountVectorizer(stop_words='english')),
    ('tfidf_scaling_stage', TfidfTransformer()),
    ('naive_bayes_model', MultinomialNB())
])
scaled_pipeline.fit(X_train, y_train)
scaled_predictions = scaled_pipeline.predict(X_test)
print("=== Task 2: Feature Transformation Variance Comparison ===")
print(f"Accuracy Matrix Configuration Without TF-IDF Scaling Transformation: {accuracy_score(y_test, baseline_predictions)*100:.2f}%")
print(f"Accuracy Matrix Configuration With Active TF-IDF Scaling Transformation : {accuracy_score(y_test, scaled_predictions)*100:.2f}%")
# Task 3: Benchmarking multiple core algorithms within identical pipeline frames
evaluation_architectures = {
    'Multinomial Naive Bayes': MultinomialNB(),
    'Logistic Regression Classifier': LogisticRegression(max_iter=1000),
    'Linear Support Vector Classifier': LinearSVC()
}
performance_benchmark_matrix = {}
for name_tag, model_instance in evaluation_architectures.items():
    bench_pipeline = Pipeline([
        ('count_vectorizer', CountVectorizer(stop_words='english')),
        ('tfidf_transformer_layer', TfidfTransformer()),
        ('classifier_estimator_node', model_instance)
    ])
    bench_pipeline.fit(X_train, y_train)
    predictions_output = bench_pipeline.predict(X_test)
    performance_benchmark_matrix[name_tag] = f"{accuracy_score(y_test, predictions_output) * 100:.2f}%"
print("\n=== Task 3: Multi-Architecture Classifier Accuracy Matrix ===")
comparison_dataframe = pd.DataFrame(list(performance_benchmark_matrix.items()), columns=['Classifier Architecture Model', 'Evaluation Accuracy Score'])
print(comparison_dataframe.to_string(index=False))

***
=== Task 1: Baseline Pipeline Performance Metrics ===
Accuracy: 97.87%

      precision    recall  f1-score   support

alt.atheism      0.99      0.99      0.99        186
rec.autos        0.98      0.98      0.98        248
sci.electronics  0.98      0.98      0.98        251
sci.med          0.97      0.98      0.97        256

accuracy          0.98
macro avg         0.98
weighted avg      0.98

=== Task 2: Feature Transformation Variance Comparison ===
Accuracy Matrix Configuration Without TF-IDF Scaling Transformation: 97.87%
Accuracy Matrix Configuration With Active TF-IDF Scaling Transformation : 97.77%

=== Task 3: Multi-Architecture Classifier Accuracy Matrix ===
Classifier Architecture Model Evaluation Accuracy Score
Multinomial Naive Bayes          97.77%
Logistic Regression Classifier    97.02%
Linear Support Vector Classifier  97.34%

```

Task 4,5

```

import tkinter as tk
from tkinter import scrolledtext, ttk
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC

# Mini Training Corpus Definitions Maps Data Structures
training_corpus_phrases = [
    'football midfielder striker score goal match league champion kick', 'basketball slam dunk court parquet
    referee hoop rebound whistle',
    'python compilation programming script function algorithm structure variable source', 'neural network deep
    learning training epoch backpropagation weights bias',
    'medicine hospital surgeon doctor patient treatment clinic therapy capsule prescription', 'surgery vaccine
    disease cure oncology diagnosis pharmacy symptoms immune',
    'election vote parliament legislative government policy supreme ballot candidate reform', 'president prime
    minister cabinet constitution law senate diplomatic ambassador budget'
]
training_domain_targets = ['Sports', 'Sports', 'Technology', 'Technology', 'Medical', 'Medical', 'Politics',
    'Politics']

active_session_pipeline = None
historical_run_traces = []

def dynamically_reconfigure_pipeline_stage(*args):
    global active_session_pipeline
    selected_estimator_string = drop_down_selector.get()

    # Task 4: Classifier switching selection routing logic
    if selected_estimator_string == 'Naive Bayes Classifier':
        estimator_instance = MultinomialNB()
    elif selected_estimator_string == 'Linear Support Vector Machine':
        estimator_instance = LinearSVC()
    else:
        estimator_instance = LogisticRegression()

    active_session_pipeline = Pipeline([
        ('tfidf_vectorization', TfidfVectorizer()),
        ('classifier_node', estimator_instance)
    ])
    active_session_pipeline.fit(training_corpus_phrases, training_domain_targets)

def process_live_pipeline_stream():
    input_text_block = text_input_field.get('1.0', tk.END).strip()
    if not input_text_block: return

    inferred_target_class = active_session_pipeline.predict([input_text_block])[0]

```

```

output_display_panel.config(state='normal')
output_display_panel.delete('1.0', tk.END)
output_display_panel.insert(tk.END, f'Selected Active Architecture Frame: {drop_down_selector.get()}\n')
output_display_panel.insert(tk.END, f'Inferred Class Target Assignment :
{inferred_target_class.upper()}\n\n')

# Check if the active algorithm supports class probability weights
if hasattr(active_session_pipeline.named_steps['classifier_node'], "predict_proba"):
    probabilities_array = active_session_pipeline.predict_proba([input_text_block])[0]
    mapped_classes_list = active_session_pipeline.classes_
    output_display_panel.insert(tk.END, "Confidence Dispersal Metric Probabilities Distribution:\n")
    for label, probability_weight in zip(mapped_classes_list, probabilities_array):
        visual_bar_string = '#' * int(probability_weight * 20)
        output_display_panel.insert(tk.END, f" {label:12s}: {visual_bar_string:20s}
({probability_weight:.3f})\n")
    else:
        output_display_panel.insert(tk.END, "Confidence Metric Probabilities Data: Not Available\n(Linear SVC
operates via hyperplanes and doesn't output probability distributions.)")
        output_display_panel.config(state='disabled')

# Task 5: Format and update the session log history
clean_snippet = input_text_block[:35].replace('\n', ' ')
log_string_entry = f' [{inferred_target_class.upper()[:4]}] -> {clean_snippet}...'

historical_run_traces.insert(0, log_string_entry)
if len(historical_run_traces) > 6:
    historical_run_traces.pop()

history_display_listbox.delete(0, tk.END)
for position_index, active_log_item in enumerate(historical_run_traces):
    history_display_listbox.insert(position_index, active_log_item)

root = tk.Tk()
root.title("Configurable Modular NLP Pipeline Deployment Interface")
root.geometry("620x660")

# Task 4: Setup dropdown selectors inside control layouts
selector_bar_panel = tk.Frame(root)
selector_bar_panel.pack(pady=10)
tk.Label(selector_bar_panel, text="Select Pipeline Classification Estimator Layer:", font=('Arial', 9,
'bold')).pack(side=tk.LEFT, padx=5)

drop_down_selector = ttk.Combobox(selector_bar_panel, values=['Logistic Regression', 'Naive Bayes
Classifier', 'Linear Support Vector Machine'], state="readonly", width=30)
drop_down_selector.set('Logistic Regression')
drop_down_selector.pack(side=tk.LEFT, padx=5)
drop_down_selector.bind('<<ComboboxSelected>>', dynamically_reconfigure_pipeline_stage)

```

```
tk.Label(root, text="Insert Raw Document Block Segment for Processing Flow:", font=('Arial', 10)).pack()
text_input_field = scrolledtext.ScrolledText(root, height=5, font=('Arial', 10))
text_input_field.pack(fill=tk.X, padx=15, pady=5)

tk.Button(root, text="Execute Processing Pipeline Flow", command=process_live_pipeline_stream, bg="black",
fg="white", font=('Arial', 10, 'bold')).pack(pady=5)

output_display_panel = scrolledtext.ScrolledText(root, height=8, font=('Courier', 10), state='disabled')
output_display_panel.pack(fill=tk.X, padx=15, pady=5)

# Task 5: Historical transaction tracking layout list box placement configurations
tk.Label(root, text="Session Processing Run Log History Frameworks (Max 6 Trace Blocks):", font=('Arial',
10, 'bold')).pack(pady=5)
history_display_listbox = tk.Listbox(root, height=6, font=('Courier', 10))
history_display_listbox.pack(fill=tk.BOTH, expand=True, padx=15, pady=10)

# Run initial pipeline bootstrap configuration sequence
dynamically_reconfigure_pipeline_stage()

root.mainloop()
```

Lab Number	Lab # 11
Title	Word Embeddings
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Understand word embedding concepts, the Word2Vec model, and use pretrained embeddings for word similarity tasks.

1. Theory

1.1 Limitations of TF-IDF

TF-IDF represents text as sparse high-dimensional vectors based on word counts. It treats each word as an independent symbol and has no notion of word meaning or semantic relationships. For example, 'car' and 'automobile' would be completely unrelated in a TF-IDF representation, despite being synonyms.

1.2 What are Word Embeddings?

Word embeddings are dense, low-dimensional vector representations of words where semantically similar words are mapped to nearby points in the vector space. They capture syntactic and semantic relationships between words.

The key idea: words that appear in similar contexts tend to have similar meanings.

For example, the vector for 'king' minus 'man' plus 'woman' approximately equals the vector for 'queen' — demonstrating that embeddings capture analogical relationships.

1.3 Word2Vec

Word2Vec is a neural network-based technique developed by Google (Mikolov et al., 2013) that learns word embeddings from a large text corpus. It has two architectures:

- CBOW (Continuous Bag of Words): Predicts a target word from surrounding context words.
- Skip-gram: Predicts surrounding context words given a target word. Better for rare words.

Training a Word2Vec model with Gensim:

```

from gensim.models import Word2Vec

sentences = [
    ['the', 'cat', 'sat', 'on', 'the', 'mat'],
    ['the', 'dog', 'sat', 'on', 'the', 'log'],
    ['the', 'cat', 'chased', 'the', 'mouse'],
    ['the', 'dog', 'chased', 'the', 'ball'],
    ['the', 'king', 'ruled', 'the', 'kingdom'],
    ['the', 'queen', 'ruled', 'the', 'land'],
]

model = Word2Vec(sentences, vector_size=50, window=3,
                  min_count=1, workers=2, epochs=100)

# Get vector for a word
print(model.wv['cat'])

# Find most similar words
print(model.wv.most_similar('cat', topn=3))

# Word similarity score (0 to 1)
print(model.wv.similarity('cat', 'dog'))

```

1.4 Using Pretrained Embeddings

Training Word2Vec requires large corpora. For most practical tasks, pretrained embeddings trained on billions of words are used. Popular choices include Google's Word2Vec (3M words, 300 dimensions) and Stanford's GloVe.

```

# Loading a small pretrained model (gensim-data)
import gensim.downloader as api

# Download a small pretrained model
wv = api.load('glove-wiki-gigaword-50') # 50-dim GloVe

print(wv.most_similar('python', topn=5))
print(wv.similarity('doctor', 'hospital'))

```



```
# Analogy: king - man + woman = ?
result = wv.most_similar(positive=['king','woman'], negative=['man'], topn=1)
print('king - man + woman =', result)
```

1.5 Tkinter Word Embeddings Explorer

```
import tkinter as tk
from tkinter import scrolledtext, messagebox
from gensim.models import Word2Vec

# Build a small demo model
corpus = [
    'cat dog mouse animal pet',
    'python java programming code software',
    'doctor hospital nurse medicine patient',
    'king queen royal palace crown',
    'football basketball sport player team',
]
tokenized = [s.split() for s in corpus * 30] # repeat for training
wv_model = Word2Vec(tokenized, vector_size=30, window=3,
                    min_count=1, epochs=200)

def find_similar():
    word = word_entry.get().strip().lower()
    try:
        similars = wv_model.wv.most_similar(word, topn=5)
        out.config(state='normal')
        out.delete('1.0', tk.END)
        out.insert(tk.END, f'Top 5 words similar to "{word}":\n\n')
        for w, score in similars:
            out.insert(tk.END, f' {w:15s} similarity = {score:.4f}\n')
        out.config(state='disabled')
    except KeyError:
        messagebox.showerror('Error', f'Word "{word}" not in vocabulary.')

def compute_similarity():
    w1 = w1_entry.get().strip().lower()
```

```

w2 = w2_entry.get().strip().lower()
try:
    sim = wv_model.wv.similarity(w1, w2)
    sim_label.config(text=f'Similarity({w1}, {w2}) = {sim:.4f}')
except KeyError as e:
    messagebox.showerror('Error', str(e))

root = tk.Tk()
root.title('Word Embeddings Explorer')
root.geometry('520x460')
tk.Label(root, text='Word Embeddings Explorer',
        font=('Arial',13,'bold')).pack(pady=8)

# Similar words frame
f1 = tk.LabelFrame(root, text='Find Similar Words', font=('Arial',10,'bold'))
f1.pack(fill=tk.X, padx=15, pady=5)
tk.Label(f1, text='Word:').grid(row=0, column=0, padx=5, pady=5)
word_entry = tk.Entry(f1, width=20); word_entry.grid(row=0, column=1, padx=5)
tk.Button(f1, text='Find Similar', command=find_similar,
        bg='black', fg='white').grid(row=0, column=2, padx=5)
out = scrolledtext.ScrolledText(root, height=7, font=('Courier',10),
        state='disabled')
out.pack(fill=tk.X, padx=15)

# Similarity frame
f2 = tk.LabelFrame(root, text='Word Similarity Score', font=('Arial',10,'bold'))
f2.pack(fill=tk.X, padx=15, pady=5)
tk.Label(f2, text='Word 1:').grid(row=0, column=0, padx=5, pady=5)
w1_entry = tk.Entry(f2, width=15); w1_entry.grid(row=0, column=1)
tk.Label(f2, text='Word 2:').grid(row=0, column=2, padx=5)
w2_entry = tk.Entry(f2, width=15); w2_entry.grid(row=0, column=3)
tk.Button(f2, text='Compute', command=compute_similarity,
        bg='black', fg='white').grid(row=0, column=4, padx=5)
sim_label = tk.Label(root, text="", font=('Arial',11,'bold'))
sim_label.pack(pady=8)
root.mainloop()

```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Train a Word2Vec model on a corpus of at least 15 sentences related to a topic of your choice (e.g., sports, technology). Print the vocabulary size and the top 5 most similar words to any 3 words from the corpus.	—
Task 2	Using your trained Word2Vec model, compute the similarity score between 5 word pairs (e.g., cat-dog, python-java). Print each pair and its similarity score sorted from highest to lowest.	—
Task 3	Modify the Tkinter Word Embeddings Explorer to also display the dimensionality of the word vector and show the actual vector values (rounded to 3 decimal places) in the output box.	—
Task 4	Add an 'Analogy Solver' frame to the Tkinter app with three entry boxes: 'Word A is to Word B as Word C is to ____'. Use <code>model.wv.most_similar(positive=[B,C], negative=[A])</code> to find the answer.	—
Task 5	Train two Word2Vec models on the same corpus but with different <code>vector_size</code> values (e.g., 10 and 100). Add a <code>RadioButton</code> to the Tkinter app to select between them and compare the similarity scores for the same word pair.	—

Task 1,2

```
from gensim.models import Word2Vec
```

```
# Task 1: Complete 15-sentence custom technology domain text corpus dictionary array
```

```
custom_technology_corpus_list = [
    ['python', 'is', 'an', 'interpreted', 'high-level', 'programming', 'language', 'for', 'general-purpose', 'development'],
    ['software', 'developers', 'write', 'clean', 'object-oriented', 'code', 'using', 'robust', 'python', 'frameworks'],
    ['machine', 'learning', 'algorithms', 'build', 'predictive', 'statistical', 'models', 'using', 'clean', 'datasets'],
    ['deep', 'learning', 'utilizes', 'artificial', 'neural', 'network', 'architectures', 'to', 'train', 'models'],
    ['cloud', 'computing', 'platforms', 'provide', 'scalable', 'virtualized', 'infrastructure', 'and', 'storage', 'nodes'],
    ['software', 'engineers', 'deploy', 'rest', 'api', 'endpoints', 'inside', 'isolated', 'docker', 'containers'],
    ['database', 'administrators', 'optimize', 'sql', 'query', 'execution', 'paths', 'using', 'index', 'keys'],
    ['cybersecurity', 'experts', 'mitigate', 'malicious', 'phishing', 'attacks', 'and', 'network', 'security', 'breaches'],
    ['version', 'control', 'systems', 'like', 'git', 'allow', 'distributed', 'code', 'collaboration', 'among', 'teams'],
    ['data', 'scientists', 'train', 'advanced', 'regression', 'models', 'to', 'forecast', 'complex', 'business', 'metrics'],
    ['artificial', 'intelligence', 'is', 'transforming', 'modern', 'automation', 'and', 'predictive', 'software', 'design'],
    ['linux', 'servers', 'host', 'critical', 'backend', 'api', 'microservices', 'with', 'high', 'availability', 'uptime'],
```

```
['frontend', 'developers', 'design', 'interactive', 'user', 'interfaces', 'with', 'modern', 'javascript', 'frameworks'],
['big', 'data', 'pipelines', 'process', 'terabytes', 'of', 'unstructured', 'log', 'information', 'every', 'hour'],
['quantum', 'computing', 'leverages', 'advanced', 'physics', 'to', 'execute', 'complex', 'mathematical', 'algorithms',
'fast']
]
```

```
# Train the Word2Vec model with custom 25-dimension vector constraints
```

```
w2v_model_instance = Word2Vec(
    sentences=custom_technology_corpus_list,
    vector_size=25,
    window=3,
    min_count=1,
    workers=1,
    epochs=250
)
```

```
print("=== Task 1: Word2Vec Structural Model Verification ===")
```

```
extracted_vocab_size = len(w2v_model_instance.wv.key_to_index)
```

```
print(f"Total Unique Vocabulary Map Items Extracted: {extracted_vocab_size}")
```

```
target_probe_tokens = ['python', 'learning', 'code']
```

```
for target_token in target_probe_tokens:
```

```
    print(f"\nTop 4 Nearest Semantic Associations for target word -> '{target_token}':")
```

```
    association_records = w2v_model_instance.wv.most_similar(target_token, topn=4)
```

```
    for token_item, similarity_score in association_records:
```

```
        print(f" Word: {token_item:15s} | Cosine Overlap Vector Distance Metric = {similarity_score:.4f}")
```

```
# Task 2: Comparative calculations across 5 custom word pairs sorted by similarity
```

```
word_pairs_to_evaluate = [
```

```
    ('python', 'code'),
```

```
    ('learning', 'algorithms'),
```

```
    ('database', 'query'),
```

```
    ('software', 'physics'),
```

```
    ('linux', 'javascript')
]
```

```
pair_similarity_matrix_log = []
```

```
for word_a, word_b in word_pairs_to_evaluate:
```

```
    computed_distance_score = w2v_model_instance.wv.similarity(word_a, word_b)
```

```
    pair_similarity_matrix_log.append((word_a, word_b, computed_distance_score))
```

```
# Sort the pairs based on the similarity metric in descending order
```

```
pair_similarity_matrix_log.sort(key=lambda item: item[2], reverse=True)
```

```
print("\n=== Task 2: Ranked Pairwise Distance Metrics Table ===")
```

```
for word_a, word_b, distance_metric in pair_similarity_matrix_log:
```

```
    print(f"Pair Matrix Relation Vector -> ({word_a:8s} <-> {word_b:12s}): Metric Similarity Correlation = {distance_metric:.4f}")
```

Tasks 3, 4 & 5

```

import tkinter as tk
from tkinter import scrolledtext, messagebox
from gensim.models import Word2Vec

# Vocabulary definitions and dataset mapping structures setup
source_synthetic_corpus = [
    'cat dog mouse animal pet fur paw tail fetch bark meow look', 'python java programming code software script
dev compile build technology',
    'doctor hospital nurse medicine patient clinic health cure therapy surgeon pharmacy', 'king queen royal palace
crown throne empire sovereign majesty princess prince',
    'football basketball sport player team match game referee league champion stadium coach'
]
tokenized_sentences_stream = [sentence_string.split() for sentence_string in source_synthetic_corpus * 35]

# Task 5: Train dual separate vector models with different dimensions
w2v_compact_dim10 = Word2Vec(tokenized_sentences_stream, vector_size=10, window=3, min_count=1,
epochs=150, seed=42)
w2v_expanded_dim100 = Word2Vec(tokenized_sentences_stream, vector_size=100, window=3, min_count=1,
epochs=150, seed=42)

def fetch_active_embedding_model_context():
    return w2v_compact_dim10 if size_selection_var.get() == "Dim_10" else w2v_expanded_dim100

def run_semantic_similarity_lookup():
    probe_word = word_search_entry.get().strip().lower()
    selected_model = fetch_active_embedding_model_context()

    try:
        closest_associations = selected_model.wv.most_similar(probe_word, topn=5)

        # Task 3: Extract and print raw weights from the vector layers
        dense_vector_array = selected_model.wv[probe_word]
        formatted_coordinates = [round(float(coordinate), 3) for coordinate in dense_vector_array]

        display_console.config(state='normal')
        display_console.delete('1.0', tk.END)
        display_console.insert(tk.END, f"Active Embedding Model Dimension Configurations:
{len(dense_vector_array)}\n")
        display_console.insert(tk.END, f"Raw Array Coordinates Data Weights:\n{formatted_coordinates}\n\n")

        display_console.insert(tk.END, f"Top 5 Closest Words vector spaces matches to \"{probe_word}\":\n")
        for matching_word, alignment_score in closest_associations:
            display_console.insert(tk.END, f" {matching_word:15s} -> Distance Cosine Fit =
{alignment_score:.4f}\n")
        display_console.config(state='disabled')

```

```

except KeyError:
    messagebox.showerror("Vocabulary Conflict", f"The token item \"{probe_word}\" is missing from the
dictionary mapping spaces.")

def calculate_pairwise_score():
    word_a = pair_word_a_entry.get().strip().lower()
    word_b = pair_word_b_entry.get().strip().lower()
    selected_model = fetch_active_embedding_model_context()

    try:
        calculated_similarity = selected_model.wv.similarity(word_a, word_b)
        pairwise_result_label.config(text=f"Calculated Metric Coordinate Linkage ({word_a} <-> {word_b}) =
{calculated_similarity:.4f}")
    except KeyError as error:
        messagebox.showerror("Mapping Core Refusal Error", str(error))

# Task 4: Algebraic analogy loop processing function
def compute_vector_analogy_target():
    w_a = analogy_input_a.get().strip().lower()
    w_b = analogy_input_b.get().strip().lower()
    w_c = analogy_input_c.get().strip().lower()
    selected_model = fetch_active_embedding_model_context()

    try:
        # Analogy formulation logic matrix operations equation -> (B - A + C)
        analogy_solutions_list = selected_model.wv.most_similar(positive=[w_b, w_c], negative=[w_a], topn=1)
        resolved_token_string, confidence_fit = analogy_solutions_list[0]
        analogy_output_lbl.config(text=f"Inferred Algebraic Analogy Target Match:
{resolved_token_string.upper()} (Score: {confidence_fit:.4f})")
    except Exception as operational_err:
        messagebox.showerror("Analogy Failure Alert", f"Could not compute algebraic path resolution
parameters: {str(operational_err)}")

root = tk.Tk()
root.title("Multi-Dimensional Word Embedding Workspace Station")
root.geometry("600x730")

# Task 5: Setup dimension variation selection radio layouts
dimension_control_frame = tk.LabelFrame(root, text="Model Vector Dimension Space Matrix Selectors
Strategy", font=('Arial', 9, 'bold'))
dimension_control_frame.pack(fill=tk.X, padx=15, pady=5)
size_selection_var = tk.StringVar(value="Dim_10")
tk.Radiobutton(dimension_control_frame, text="Compact Vector Space (Dimension = 10)",
variable=size_selection_var, value="Dim_10").pack(side=tk.LEFT, padx=12)
tk.Radiobutton(dimension_control_frame, text="Expanded Vector Space (Dimension = 100)",
variable=size_selection_var, value="Dim_100").pack(side=tk.LEFT, padx=12)

lookup_frame = tk.LabelFrame(root, text="Find Semantically Similar Coordinate Points", font=('Arial', 9,

```

```

'bold'))
lookup_frame.pack(fill=tk.X, padx=15, pady=5)
tk.Label(lookup_frame, text="Target Key Term:").grid(row=0, column=0, padx=5, pady=5)
word_search_entry = tk.Entry(lookup_frame, width=20); word_search_entry.grid(row=0, column=1, padx=5)
tk.Button(lookup_frame, text="Run Vector Map Discovery", command=run_semantic_similarity_lookup,
bg="black", fg="white").grid(row=0, column=2, padx=5)

display_console = scrolledtext.ScrolledText(root, height=10, font=('Courier', 9), state='disabled')
display_console.pack(fill=tk.X, padx=15, pady=5)

pairwise_frame = tk.LabelFrame(root, text="Pairwise Vector Coordinate Distance Calculators", font=('Arial', 9,
'bold'))
pairwise_frame.pack(fill=tk.X, padx=15, pady=5)
tk.Label(pairwise_frame, text="Word Alpha:").grid(row=0, column=0, padx=5, pady=5)
pair_word_a_entry = tk.Entry(pairwise_frame, width=12); pair_word_a_entry.grid(row=0, column=1)
tk.Label(pairwise_frame, text="Word Beta:").grid(row=0, column=2, padx=5)
pair_word_b_entry = tk.Entry(pairwise_frame, width=12); pair_word_b_entry.grid(row=0, column=3)
tk.Button(pairwise_frame, text="Compute Similarity", command=calculate_pairwise_score, bg="black",
fg="white").grid(row=0, column=4, padx=5)

pairwise_result_label = tk.Label(root, text="", font=('Arial', 10, 'bold'), fg="darkmagenta")
pairwise_result_label.pack(pady=4)

# Task 4: Structural layout design implementation for interactive vector analogy components
analogy_workspace_frame = tk.LabelFrame(root, text="Vector Space Algebraic Analogy Matrix Solver",
font=('Arial', 9, 'bold'))
analogy_workspace_frame.pack(fill=tk.X, padx=15, pady=5)
tk.Label(analogy_workspace_frame, text="Word A:").grid(row=0, column=0, padx=2, pady=5)
analogy_input_a = tk.Entry(analogy_workspace_frame, width=10); analogy_input_a.grid(row=0, column=1)
tk.Label(analogy_workspace_frame, text="is to Word B:").grid(row=0, column=2, padx=2)
analogy_input_b = tk.Entry(analogy_workspace_frame, width=10); analogy_input_b.grid(row=0, column=3)
tk.Label(analogy_workspace_frame, text="As Word C:").grid(row=0, column=4, padx=2)
analogy_input_c = tk.Entry(analogy_workspace_frame, width=10); analogy_input_c.grid(row=0, column=5)

tk.Button(analogy_workspace_frame, text="Solve Analogy Equation",
command=compute_vector_analogy_target, bg="blue", fg="white").grid(row=1, column=0, columnspan=6,
pady=5)
analogy_output_lbl = tk.Label(analogy_workspace_frame, text="Inferred Algebraic Analogy Target Match:
Clear", font=('Arial', 10, 'italic', 'bold'), fg="darkgreen")
analogy_output_lbl.grid(row=2, column=0, columnspan=6, pady=2)

root.mainloop()

```

Lab # 12: Mini NLP Project

Lab Number	Lab # 12
Title	Mini NLP Project
Duration	1 Hour
Tools	Python 3.x, Tkinter, NLTK, Scikit-learn, Gensim
Objectives	Build a complete NLP application covering text cleaning, feature extraction, model training, and result analysis with a Tkinter GUI.

1. Theory

1.1 The Complete NLP Pipeline

A production-grade NLP system integrates all the steps learned in previous labs into one coherent pipeline. This lab brings together text cleaning, TF-IDF feature extraction, machine learning classification, and evaluation within a single interactive desktop application.

The four stages of the mini project:

- Stage 1 – Text Cleaning: Normalize, tokenize, remove noise.
- Stage 2 – Feature Extraction: Convert cleaned text to numerical features.
- Stage 3 – Model Training: Train a classifier and persist it.
- Stage 4 – Testing and Result Analysis: Evaluate, visualize, and interpret.

1.2 Stage 1: Text Cleaning

```
import re, string
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import nltk
nltk.download(['stopwords','punkt'], quiet=True)

stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
```



```

text = re.sub(r'http\S+', '', text)    # remove URLs
text = re.sub(r'[^\a-z\s]', '', text)  # keep letters only
tokens = text.split()
tokens = [t for t in tokens if t not in stop_words]
tokens = [stemmer.stem(t) for t in tokens] # stemming
return ' '.join(tokens)

```

1.3 Stage 2: Feature Extraction

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=3000, ngram_range=(1,2))
# ngram_range=(1,2) captures unigrams AND bigrams
# max_features=3000 keeps only the top 3000 features

```

1.4 Stage 3: Model Training and Saving

```

from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
import pickle

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=3000, ngram_range=(1,2))),
    ('clf', LogisticRegression(max_iter=300))
])

pipeline.fit(X_train, y_train)

# Save model for reuse
with open('nlp_model.pkl', 'wb') as f:
    pickle.dump(pipeline, f)

# Load model
with open('nlp_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)

```

1.5 Stage 4: Evaluation and Analysis

```
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)
import matplotlib.pyplot as plt
```

```
y_pred = pipeline.predict(X_test)
```

```
print('=== Model Evaluation Report ===')
print(f'Accuracy : {accuracy_score(y_test,y_pred)*100:.2f}%')
print(classification_report(y_test, y_pred))
```

```
# Cross-validation
```

```
scores = cross_val_score(pipeline, X_all, y_all, cv=5)
print(f'Cross-Val Accuracy: {scores.mean()*100:.2f}% +/- {scores.std()*100:.2f}%')
```

```
# Confusion matrix plot
```

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot(cmap='Greys')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=100)
plt.show()
```

1.6 Complete Tkinter Mini NLP Project App

The following is a complete Tkinter application integrating all four stages:

```
import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
import nltk
nltk.download(['stopwords'], quiet=True)
```

```

stemmer = PorterStemmer()
stop_w = set(stopwords.words('english'))

def clean(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    toks = [stemmer.stem(t) for t in text.split() if t not in stop_w]
    return ' '.join(toks)

```

— Labelled Training Dataset

```

raw_data = [
    ('Great product works perfectly love it', 'positive'),
    ('Amazing quality fast delivery happy', 'positive'),
    ('Excellent service friendly staff', 'positive'),
    ('Best purchase highly recommend this', 'positive'),
    ('Terrible waste of money broken', 'negative'),
    ('Awful product stopped working fast', 'negative'),
    ('Very disappointed poor quality', 'negative'),
    ('Worst experience ever do not buy', 'negative'),
    ('Average nothing special okay', 'neutral'),
    ('Received item as described expected', 'neutral'),
]

```

```

texts = [clean(r[0]) for r in raw_data]
labels = [r[1] for r in raw_data]
X_tr, X_te, y_tr, y_te = train_test_split(texts, labels,
                                          test_size=0.3, random_state=42)

```

```

model = Pipeline([('tfidf', TfidfVectorizer()),
                  ('clf', LogisticRegression(max_iter=300))])
model.fit(X_tr, y_tr)
y_pred = model.predict(X_te)
acc = accuracy_score(y_te, y_pred)
report = classification_report(y_te, y_pred)

```

— GUI

```
root = tk.Tk()
root.title('Mini NLP Project – Review Classifier')
root.geometry('620x540')

nb = ttk.Notebook(root)
nb.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
```

— Tab 1: Predict

```
tab1 = ttk.Frame(nb); nb.add(tab1, text=' Predict ')
tk.Label(tab1, text='Enter review text:',
         font=('Arial',10,'bold')).pack(pady=5)
inp = scrolledtext.ScrolledText(tab1, height=5, font=('Arial',10))
inp.pack(fill=tk.X, padx=10)

pred_var = tk.StringVar()
conf_var = tk.StringVar()

def predict():
    raw = inp.get('1.0', tk.END).strip()
    if not raw: return
    cleaned = clean(raw)
    pred = model.predict([cleaned])[0]
    probs = model.predict_proba([cleaned])[0]
    classes = model.classes_
    pred_var.set(f'Prediction: {pred.upper()}')
    conf_var.set(' | '.join(f'{c}: {p:.2f}'
                           for c,p in zip(classes,probs)))
    cleaned_var.set(f'Cleaned: {cleaned}')

tk.Button(tab1, text='Classify', command=predict,
         bg='black', fg='white', font=('Arial',10,'bold')).pack(pady=6)
tk.Label(tab1, textvariable=pred_var, font=('Arial',12,'bold')).pack()
tk.Label(tab1, textvariable=conf_var, font=('Arial',10)).pack()
cleaned_var = tk.StringVar()
```

```
tk.Label(tab1, textvariable=cleaned_var,
        font=('Arial',9,'italic'), fg='gray').pack(pady=4)
```

```
# — Tab 2: Model Report
```

```
tab2 = ttk.Frame(nb); nb.add(tab2, text=' Model Report ')
rep_box = scrolledtext.ScrolledText(tab2, font=('Courier',10))
rep_box.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
rep_box.insert(tk.END, f'=== NLP MODEL EVALUATION ===\n\n')
rep_box.insert(tk.END, f'Training samples : {len(X_tr)}\n')
rep_box.insert(tk.END, f'Test samples    : {len(X_te)}\n')
rep_box.insert(tk.END, f'Test Accuracy   : {acc*100:.2f}%\n\n')
rep_box.insert(tk.END, 'Classification Report:\n')
rep_box.insert(tk.END, report)
rep_box.config(state='disabled')
```

```
# — Tab 3: Training Data
```

```
tab3 = ttk.Frame(nb); nb.add(tab3, text=' Training Data ')
cols = ('Text', 'Label')
tree = ttk.Treeview(tab3, columns=cols, show='headings', height=14)
for c in cols: tree.heading(c, text=c); tree.column(c, width=260)
for txt, lbl in raw_data: tree.insert("", tk.END, values=(txt, lbl))
tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

root.mainloop()
```

2. Student Tasks

Task #	Task Description	Marks
Task 1	Extend the training dataset to at least 20 labeled samples (at least 6 per class). Retrain the model and print the updated accuracy and classification report in the Model Report tab.	—
Task 2	Add a 'Clean Text Preview' button next to the input in the Predict tab that shows the cleaned/preprocessed version of the entered text in a separate label or popup before prediction.	—

Task 3	Add a fourth tab called 'Batch Predict' where the user can paste multiple texts (one per line) in a ScrolledText box, click 'Classify All', and see a table of text snippets with their predicted labels.	_____
Task 4	Implement a simple confusion matrix display in the Model Report tab. Show it as a formatted text table using counts from <code>sklearn.metrics.confusion_matrix</code> , with rows as actual labels and columns as predicted labels.	_____
Task 5	Add a 'Save Report' button to the Model Report tab that writes the full evaluation report (accuracy, classification report, date and time) to a plain text file called 'nlp_report.txt' in the current directory.	_____

```

import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox
import re
from datetime import datetime
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
import nltk

# Initialize baseline structural components configuration references
nltk.download('stopwords', quiet=True)
stemming_utility = PorterStemmer()
english_stopwords = set(stopwords.words('english'))

def centralized_cleaning_pipeline(raw_text_stream):
    lowered_string = raw_text_stream.lower()
    alphabetic_only_string = re.sub(r'^a-z\s', '', lowered_string)
    tokenized_stems = [stemming_utility.stem(token) for token in alphabetic_only_string.split() if token not in english_stopwords]
    return ' '.join(tokenized_stems)

# Task 1: Expanded balanced 21-element training dataset repository matrix
master_training_data_source = [
    ('Great product works perfectly love it', 'positive'),
    ('Amazing quality fast delivery happy', 'positive'),
    ('Excellent service friendly staff', 'positive'),
    ('Best purchase highly recommend this item', 'positive'),
    ('Fantastic performance super easy interface', 'positive'),
    ('Wonderful experience absolutely beautiful design', 'positive'),

```

```

('Incredible speed exceeded my expectations completely', 'positive'),

('Terrible waste of money broken piece', 'negative'),
('Awful product stopped working fast defect', 'negative'),
('Very disappointed poor quality garbage materials', 'negative'),
('Worst experience ever do not buy nightmare', 'negative'),
('Horrible customer service representative refused refund', 'negative'),
('Useless application crashes repeatedly error failure', 'negative'),
('Broke within twenty minutes completely hate it', 'negative'),

('Average nothing special okay performance', 'neutral'),
('Received item as described expected simple', 'neutral'),
('Standard operational output unremarkable functional normal', 'neutral'),
('Fair quality matching price baseline parameters', 'neutral'),
('Mediocre result neither good nor bad standard', 'neutral'),
('As expected generic packaging routine item arrival', 'neutral'),
('It works but satisfies only fundamental regular conditions', 'neutral')
]

# Run the dataset through the normalization step
preprocessed_documents = [centralized_cleaning_pipeline(sample[0]) for sample in
master_training_data_source]
categorical_target_labels = [sample[1] for sample in master_training_data_source]

# Split the dataset using fixed seed values
X_train, X_test, y_train, y_test = train_test_split(
    preprocessed_documents, categorical_target_labels, test_size=0.30, random_state=42
)

# Build and train the unified core processing pipeline
nlp_classifier_pipeline = Pipeline([
    ('tfidf_vectorizer', TfidfVectorizer()),
    ('logistic_estimator', LogisticRegression(max_iter=400))
])
nlp_classifier_pipeline.fit(X_train, y_train)

# Generate baseline classification performance metrics
test_set_predictions = nlp_classifier_pipeline.predict(X_test)
calculated_accuracy_score = accuracy_score(y_test, test_set_predictions)
textual_classification_report = classification_report(y_test, test_set_predictions, zero_division=0)

root = tk.Tk()
root.title("Integrated Mini NLP Evaluation Platform Workstation")
root.geometry("740x660")

notebook_container = ttk.Notebook(root)
notebook_container.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

# --- TAB 1: PREDICT ENVIRONMENT WORKSPACE PANEL LAYOUT ---

```

```

prediction_tab_frame = tk.Frame(notebook_container)
notebook_container.add(prediction_tab_frame, text=' Prediction Deployment Workspace ')

tk.Label(prediction_tab_frame, text='Input Live Feedback Text Document Block:', font=('Arial', 10,
'bold')).pack(pady=5)
live_text_input = scrolledtext.ScrolledText(prediction_tab_frame, height=6, font=('Arial', 10))
live_text_input.pack(fill=tk.X, padx=15)

# String trace references variable tracking structures
class_verdict_var = tk.StringVar(value="Classification Decision Target: Pending Input")
confidence_metrics_var = tk.StringVar()
pipeline_trace_var = tk.StringVar()

def execute_live_inference_prediction():
    raw_input_content = live_text_input.get('1.0', tk.END).strip()
    if not raw_input_content: return

    cleaned_variant = centralized_cleaning_pipeline(raw_input_content)
    assigned_class_verdict = nlp_classifier_pipeline.predict([cleaned_variant])[0]
    class_probability_distributions = nlp_classifier_pipeline.predict_proba([cleaned_variant])[0]
    pipeline_classes_list = nlp_classifier_pipeline.classes_

    class_verdict_var.set(f'Classification Decision Target: {assigned_class_verdict.upper()}')
    confidence_metrics_var.set(' | '.join(f'{lbl.upper()}: {weight:.2f}' for lbl, weight in zip(pipeline_classes_list,
class_probability_distributions)))
    pipeline_trace_var.set(f'Preprocessed Pipeline Variant: \"{cleaned_variant}\"')

# Task 2: Implement cleaning lookup popup preview callbacks
def trigger_preprocessing_popup_preview():
    raw_document_content = live_text_input.get('1.0', tk.END).strip()
    if not raw_document_content:
        messagebox.showinfo("Preview Information Block", "The text entry workspace configuration is currently
empty.")
        return
    cleaned_output_preview = centralized_cleaning_pipeline(raw_document_content)
    messagebox.showinfo("Pipeline Preprocessing Execution Preview Window",
        f'Original Data Input String Vector:\n\"{raw_document_content}\"\\n\\nNormalized Token
Variant Transformations:\n\"{cleaned_output_preview}\"')

action_buttons_panel = tk.Frame(prediction_tab_frame)
action_buttons_panel.pack(pady=10)

tk.Button(action_buttons_panel, text='Classify Text Stream', command=execute_live_inference_prediction,
bg="black", fg="white", font=('Arial', 10, 'bold')).pack(side=tk.LEFT, padx=5)
# Task 2: Place text preview trigger button onto user layouts
tk.Button(action_buttons_panel, text='Clean Text Preview', command=trigger_preprocessing_popup_preview,
bg="blue", fg="white", font=('Arial', 10)).pack(side=tk.LEFT, padx=5)

tk.Label(prediction_tab_frame, textvariable=class_verdict_var, font=('Arial', 11, 'bold'),

```



```

fg="darkblue").pack(pady=5)
tk.Label(prediction_tab_frame, textvariable=confidence_metrics_var, font=('Courier', 10, 'bold'),
fg="purple").pack(pady=2)
tk.Label(prediction_tab_frame, textvariable=pipeline_trace_var, font=('Arial', 9, 'italic'),
fg="dimgray").pack(pady=5)

# --- TAB 2: MODEL REPORT GENERATION BENCHMARKS SUMMARY WORKSPACE PANEL ---
report_tab_frame = ttk.Frame(notebook_container)
notebook_container.add(report_tab_frame, text=' Performance Evaluation Report ')

report_display_console = scrolledtext.ScrolledText(report_tab_frame, font=('Courier', 10))
report_display_console.pack(fill=tk.BOTH, expand=True, padx=15, pady=5)

def populate_analytical_report_console():
    report_display_console.config(state='normal')
    report_display_console.delete('1.0', tk.END)
    report_display_console.insert(tk.END, f'=== METRIC ENGINE ANALYSIS VALIDATION
EVALUATION ===\n\n')
    report_display_console.insert(tk.END, f'Total Training Records Count Pool: {len(X_train)}\n')
    report_display_console.insert(tk.END, f'Total Testing Records Count Pool : {len(X_test)}\n')
    report_display_console.insert(tk.END, f'Model General Matrix Accuracy Fit: {calculated_accuracy_score *
100:.2f}%\n\n')
    report_display_console.insert(tk.END, f'Classification Report Text Analytics Matrix
Breakdown:\n{textual_classification_report}\n')

# Task 4: Calculate and format the confusion matrix as a raw text table
report_display_console.insert(tk.END, "Confusion Evaluation Accuracy Dispersal Table Matrix View:\n")
sorted_unique_labels = sorted(list(set(categorical_target_labels)))
computed_confusion_matrix = confusion_matrix(y_test, test_set_predictions, labels=sorted_unique_labels)

table_header_string = f'      | " + " | ".join(f'{lbl[:4].upper():4s}' for lbl in sorted_unique_labels)
report_display_console.insert(tk.END, table_header_string + "\n" + "-" * len(table_header_string) + "\n")
for index_row, row_label in enumerate(sorted_unique_labels):
    matrix_row_values_string = " | ".join(f'{count_value:4d}' for count_value in
computed_confusion_matrix[index_row])
    report_display_console.insert(tk.END, f'{row_label[:9]:9s} | {matrix_row_values_string}\n')

# Task 5: Document generation utility transaction handler block logic
def export_analytical_report_to_disk():
    text_report_content = report_display_console.get('1.0', tk.END)
    system_timestamp_string = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    try:
        with open("nlp_evaluation_report.txt", "w") as file_writer:
            file_writer.write(f'Report Generation Transaction Log Timestamp Anchor:
{system_timestamp_string}\n')
            file_writer.write(text_report_content)
            messagebox.showinfo("Disk Write Success Notice", "Full validation report saved to
'nlp_evaluation_report.txt'.")

```

```

except Exception as io_error:
    messagebox.showerror("Disk Write Exception Error", f"File write transaction failed: {str(io_error)}")

tk.Button(report_tab_frame, text="Save Analysis Report Artifact File to Disk Workspace",
command=export_analytical_report_to_disk, bg="darkgreen", fg="white", font=('Arial', 10,
'bold')).pack(pady=5)
populate_analytical_report_console()
report_display_console.config(state='disabled')

# --- TAB 3: MASTER ENGINE TRAIN SEED MATRIX VIEW PANEL ---
repository_tab_frame = tk.Frame(notebook_container)
notebook_container.add(repository_tab_frame, text=' Repository Corpus Source Registry ')

table_columns_tuple = ('Source Raw Text Document Sequences String Block', 'Associated Labeled Ground
Truth Class Category')
database_tree_widget = ttk.Treeview(repository_tab_frame, columns=table_columns_tuple, show='headings')
database_tree_widget.heading(table_columns_tuple[0], text=table_columns_tuple[0])
database_tree_widget.heading(table_columns_tuple[1], text=table_columns_tuple[1])
database_tree_widget.column(table_columns_tuple[0], width=450)
database_tree_widget.column(table_columns_tuple[1], width=150)

for sample_phrase, source_class_label in master_training_data_source:
    database_tree_widget.insert("", tk.END, values=(sample_phrase, source_class_label))
database_tree_widget.pack(fill=tk.BOTH, expand=True, padx=15, pady=15)

# --- TAB 4: BATCH PROCESSING PIPELINE AUTOMATION CONTROL BLOCK INTERFACE ---
# Task 3: Interactive multi-line inputs stream workspace layout view panel
batch_predict_tab_frame = tk.Frame(notebook_container)
notebook_container.add(batch_predict_tab_frame, text=' Batch Stream Processing Engine ')

tk.Label(batch_predict_tab_frame, text="Paste Multiple Custom Texts (One Document Target Per Single Row
Line Matrix Entry):", font=('Arial', 10, 'bold')).pack(pady=5)
batch_input_workspace = scrolledtext.ScrolledText(batch_predict_tab_frame, height=8, font=('Arial', 10))
batch_input_workspace.pack(fill=tk.X, padx=15, pady=5)

def run_automated_batch_classification_sequence():
    raw_block_data = batch_input_workspace.get('1.0', tk.END).strip()
    if not raw_block_data:
        messagebox.showwarning("Batch Execution Fault Warning", "The interactive input workspace area
contains no text rows.")
        return

    # Clear out older data fields rows inside view displays
    for active_item in batch_results_tree.get_children():
        batch_results_tree.delete(active_item)

    line_row_records = [line_row.strip() for line_row in raw_block_data.split('\n') if line_row.strip()]

```

```

for single_record_line in line_row_records:
    preprocessed_line_variant = centralized_cleaning_pipeline(single_record_line)
    if not preprocessed_line_variant:
        batch_inferred_class_output = "NEUTRAL (FALLBACK)"
    else:
        batch_inferred_class_output = nlp_classifier_pipeline.predict([preprocessed_line_variant])[0].upper()

    truncated_snippet_string = single_record_line if len(single_record_line) <= 52 else
    f"{single_record_line[:49]}..."
    batch_results_tree.insert("", tk.END, values=(truncated_snippet_string, batch_inferred_class_output))

tk.Button(batch_predict_tab_frame, text="Run Batch Transformation Analysis Sequence",
command=run_automated_batch_classification_sequence, bg="purple", fg="white", font=('Arial', 10,
'bold')).pack(pady=5)

batch_table_columns_tuple = ('Sample Text Row Fragment Snippet Content Matrix Display', 'Automated
Predicted Label Target Assignment')
batch_results_tree = ttk.Treeview(batch_predict_tab_frame, columns=batch_table_columns_tuple,
show='headings', height=10)
batch_results_tree.heading(batch_table_columns_tuple[0], text=batch_table_columns_tuple[0])
batch_results_tree.heading(batch_table_columns_tuple[1], text=batch_table_columns_tuple[1])
batch_results_tree.column(batch_table_columns_tuple[0], width=440)
batch_results_tree.column(batch_table_columns_tuple[1], width=180)
batch_results_tree.pack(fill=tk.BOTH, expand=True, padx=15, pady=5)

root.mainloop()

```